

DOCKER TASK - 03

1. Create a Docker image from running container.

1. Objective

The objective is to create a new Docker image from an existing/running Docker container using the docker commit command.

The basic flow is:

Running Container → docker commit → New Docker Image

2. Check the Running Containers

First, check which containers are currently running.

docker ps

Example:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
PORTS	NAMES			
cb9a128373f3	my-nginx-image	"nginx -g 'daemon of...'"	1 hour ago	Up 1 hour
0.0.0.0:80->80/tcp	my-nginx-container			

Here:

- **Container ID:** cb9a128373f3
- **Container Name:** my-nginx-container
- **Current Image:** my-nginx-image

```
root@ip-172-31-30-131:~/my-nginx
[root@ip-172-31-30-131 my-nginx]# docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
my-nginx-image      v1              39c541b70324   5 minutes ago  162MB
tomcat               10.1           29de696ad660   4 days ago     414MB
nginx                latest          f075e3f94986   6 days ago     162MB
[root@ip-172-31-30-131 my-nginx]# docker run -d --name my-nginx-container -p 80:80 my-nginx-image:v1
93afbe5f6449e3e4c01a498b90e5bdd6d7ab4f1dc93af56e74912ab17a274b1e
[root@ip-172-31-30-131 my-nginx]# docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED            STATUS              PORTS
93afbe5f6449        my-nginx-image:v1  "/docker-entrypoint..." 9 seconds ago      Up 8 seconds       0.0.0.0:80->80/tcp
, ::80->80/tcp      my-nginx-container
[root@ip-172-31-30-131 my-nginx]#
```

3. Make Changes Inside the Running Container

Enter the running container.

```
docker exec -it my-nginx-container /bin/bash
```

If Bash is not available, use:

```
docker exec -it my-nginx-container /bin/sh
```

For example, check the Nginx web files:

```
cd /usr/share/nginx/html
```

```
ls
```

You can create or modify a file:

```
echo "This is my modified Nginx container" > /usr/share/nginx/html/index.html
```

Verify the change:

```
cat /usr/share/nginx/html/index.html
```

Exit the container:

```
Exit
```

```
root@ip-172-31-30-131:~/my-nginx
[root@ip-172-31-30-131 my-nginx]# docker exec -it my-nginx-container /bin/bash
root@93afbe5f6449:/# ls
bin      dev                docker-entrypoint.sh  home  lib64  mnt  proc  run  srv  tmp  var
boot    docker-entrypoint.d  etc                  lib   media  opt  root  sbin  sys  usr
root@93afbe5f6449:/# cd /usr/share/nginx/html
root@93afbe5f6449:/usr/share/nginx/html# ls
50x.html  index.html
root@93afbe5f6449:/usr/share/nginx/html# cat index.html
<html>
<head>
  <title>My Custom Docker Image</title>
</head>
<body>
  <h1>Hello from Rajkumari's Docker Image!</h1>
</body>
</html>
root@93afbe5f6449:/usr/share/nginx/html# |
```

4. Verify the Container

Check that the container is still running:

```
docker ps
```

You can also test the application from the Docker host:

```
curl http://localhost
```

You should see the modified HTML content.



Hello from Rajkumari's Docker Image!

5. Create an Image from the Running Container

Now use the docker commit command.

Syntax

```
docker commit <container-name-or-id> <new-image-name>:<tag>
```

Example

```
docker commit my-nginx-container my-nginx-backup:v1
```

Output will look similar to:

```
sha256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

This means Docker successfully created a new image from the container.

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# docker ps  
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS  
338e37111bc6   my-nginx-image:v1  "/docker-entrypoint..."  4 minutes ago  Up 4 minutes  0.0.0.0:80->80/tcp  
[root@ip-172-31-30-131 ~]# docker commit my-nginx-container my-nginx-running:v2  
sha256:384c40c044632f913ee517d8481e7c93d156fec059d3ef69a3798768efeca1d5  
[root@ip-172-31-30-131 ~]# docker images  
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE  
my-nginx-running  v2        384c40c04463   14 seconds ago  162MB  
my-nginx-image    v1        39c541b70324   2 hours ago    162MB  
tomcat           10.1      29de696ad660   4 days ago     414MB  
nginx             latest    f075e3f94986   6 days ago     162MB  
[root@ip-172-31-30-131 ~]#
```

6. Verify the New Image

Run:

docker images

The newly created image is: docker commit <Container name or ID> <New Image name>: version

my-nginx-running:v2

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# docker ps  
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS  
338e37111bc6   my-nginx-image:v1  "/docker-entrypoint..."  4 minutes ago  Up 4 minutes  0.0.0.0:80->80/tcp  
[root@ip-172-31-30-131 ~]# docker commit my-nginx-container my-nginx-running:v2  
sha256:384c40c044632f913ee517d8481e7c93d156fec059d3ef69a3798768efeca1d5  
[root@ip-172-31-30-131 ~]# docker images  
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE  
my-nginx-running  v2        384c40c04463   14 seconds ago  162MB  
my-nginx-image    v1        39c541b70324   2 hours ago    162MB  
tomcat           10.1      29de696ad660   4 days ago     414MB  
nginx             latest    f075e3f94986   6 days ago     162MB  
[root@ip-172-31-30-131 ~]#
```

7. Run a New Container from the Created Image

Create a new container using the new image:

docker run -d --name nginx-run-cont -p 8080:80 my-nginx-running:v2

Check the container:

docker ps

You should see:

CONTAINER ID	IMAGE	PORTS	NAMES
xxxxxxx	my-nginx-running:v1	0.0.0.0:81->80/tcp	nginx-backup-container

```
[root@ip-172-31-30-131 ~]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
my-nginx-running     v2          384c40c04463 4 minutes ago  162MB
my-nginx-image       v1          39c541b70324 2 hours ago    162MB
tomcat               10.1       29de696ad660 4 days ago     414MB
nginx                latest      f075e3f94986 6 days ago     162MB
[root@ip-172-31-30-131 ~]# docker run -d --name my-nginx-run-cont -p 81:80 my-nginx-running:v2
da4916be1c8685c2dd70b4c91e034ccc0655c66e4e7cd658b52c1ce2e097d538
[root@ip-172-31-30-131 ~]#
```

8. Test the New Container

Use:

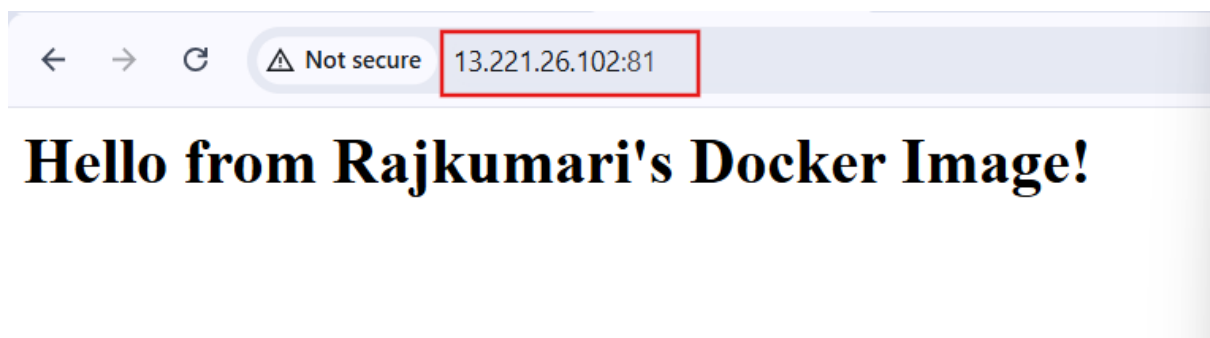
```
curl http://localhost:8080
```

You should receive the HTML content that was present in the original container when the image was committed.

You can also access it from a browser:

```
http://<EC2-Public-IP>:81
```

Make sure port **81** is allowed in the EC2 Security Group if accessing it from your computer.



9. Inspect the Created Image

You can inspect the image using:

```
docker inspect my-nginx-backup:v1
```

This displays information such as:

- Image ID
- Created time
- Architecture
- Environment variables

- Entrypoint
- Command
- Root filesystem
- Container configuration

```
root@ip-172-31-30-131:~# docker inspect my-nginx-running:v2
[{"Id": "sha256:384c40c044632f913ee517d8481e7c93d156fec059d3ef69a3798768efeca1d5",
  "RepoTags": [
    "my-nginx-running:v2"
  ],
  "RepoDigests": [],
  "Parent": "sha256:39c541b70324a85599f81589ed2471dae1e29818be8140c83fe6fc7d9457e34b",
  "Comment": "",
  "Created": "2026-08-26T14:17:46.849871557Z",
  "Container": "338e37111bc6982c06008728ed73929987d9c28da59d7b12e9aa1cbb2b458d",
  "ContainerConfig": {
    "Hostname": "338e37111bc6",
    "Domainname": "",
    "User": "",
    "AttachStdin": false,
    "AttachStdout": false,
    "AttachStderr": false,
    "ExposedPorts": {
      "80/tcp": {}
    },
    "TTY": false,
    "OpenStdin": false,
    "StdinOnce": false,
    "Env": [
      "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
      "NGINX_VERSION=1.31.4",
      "NJS_VERSION=1.0.0",
      "NJS_RELEASE=1~trixie",
      "ACME_VERSION=0.4.1",
      "PKG_RELEASE=1~trixie",
      "DYNPKG_RELEASE=1~trixie"
    ],
    "Cmd": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "Image": "my-nginx-image:v1",
    "Volumes": null,
    "WorkingDir": "",
    "Entrypoint": [
      "/docker-entrypoint.sh"
    ],
    "OnBuild": null,
    "Labels": {
      "maintainer": "NGINX Docker Maintainers <docker-maint@nginx.com>"
    },
    "StopSignal": "SIGQUIT"
  },
  "DockerVersion": "25.0.16",
  "Author": "",
  "Config": {
    "Hostname": "338e37111bc6",
    "Domainname": "",
    "User": "",
    "AttachStdin": false,
```

10. View the Image History

Use:

`docker history my-nginx-running:v2`

This shows the layers and history associated with the image.

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# docker history my-nginx-running:v2
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
384c40c04463	11 minutes ago	nginx -g daemon off;	1.09kB	
39c541b70324	2 hours ago	CMD ["nginx" "-g" "daemon off;"]	0B	buildkit.dockerfile.v0
<missing>	2 hours ago	EXPOSE map[80/tcp:{}]	0B	buildkit.dockerfile.v0
<missing>	2 hours ago	COPY index.html /usr/share/nginx/html/index...	137B	buildkit.dockerfile.v0
<missing>	6 days ago	CMD ["nginx" "-g" "daemon off;"]	0B	buildkit.dockerfile.v0
<missing>	6 days ago	STOPSIGNAL SIGQUIT	0B	buildkit.dockerfile.v0
<missing>	6 days ago	EXPOSE map[80/tcp:{}]	0B	buildkit.dockerfile.v0
<missing>	6 days ago	ENTRYPOINT ["/docker-entrypoint.sh"]	0B	buildkit.dockerfile.v0
<missing>	6 days ago	COPY 30-tune-worker-processes.sh /docker-ent...	4.62kB	buildkit.dockerfile.v0
<missing>	6 days ago	COPY 20-envsubst-on-templates.sh /docker-ent...	3.03kB	buildkit.dockerfile.v0
<missing>	6 days ago	COPY 15-local-resolvers.envsh /docker-entryp...	389B	buildkit.dockerfile.v0
<missing>	6 days ago	COPY 10-listen-on-ipv6-by-default.sh /docker...	2.12kB	buildkit.dockerfile.v0
<missing>	6 days ago	COPY docker-entrypoint.sh / # buildkit	1.62kB	buildkit.dockerfile.v0
<missing>	6 days ago	RUN /bin/sh -c set -x && groupadd --syst...	83.2MB	buildkit.dockerfile.v0
<missing>	6 days ago	ENV DYNPKG_RELEASE=1~trixie	0B	buildkit.dockerfile.v0
<missing>	6 days ago	ENV PKG_RELEASE=1~trixie	0B	buildkit.dockerfile.v0
<missing>	6 days ago	ENV ACME_VERSION=0.4.1	0B	buildkit.dockerfile.v0
<missing>	6 days ago	ENV NJS_RELEASE=1~trixie	0B	buildkit.dockerfile.v0
<missing>	6 days ago	ENV NJS_VERSION=1.0.0	0B	buildkit.dockerfile.v0
<missing>	6 days ago	ENV NGINX_VERSION=1.31.4	0B	buildkit.dockerfile.v0
<missing>	6 days ago	LABEL maintainer=NGINX Docker Maintainers <d...	0B	buildkit.dockerfile.v0
<missing>	3 weeks ago	# debian.sh --arch 'amd64' out/ 'trixie' '@1...	78.6MB	debuerreotype 0.17

```
[root@ip-172-31-30-131 ~]#
```

2.Copy Docker image from local machine to docker server and load the image.

Local Machine

```
|  
| docker save
```

↓

Docker Image (.tar file)

```
|  
| scp
```

↓

Remote Docker Server

```
|  
| docker load
```

↓

Docker Image available on server

2. Check the Image on the Local Machine

First, list the available Docker images:

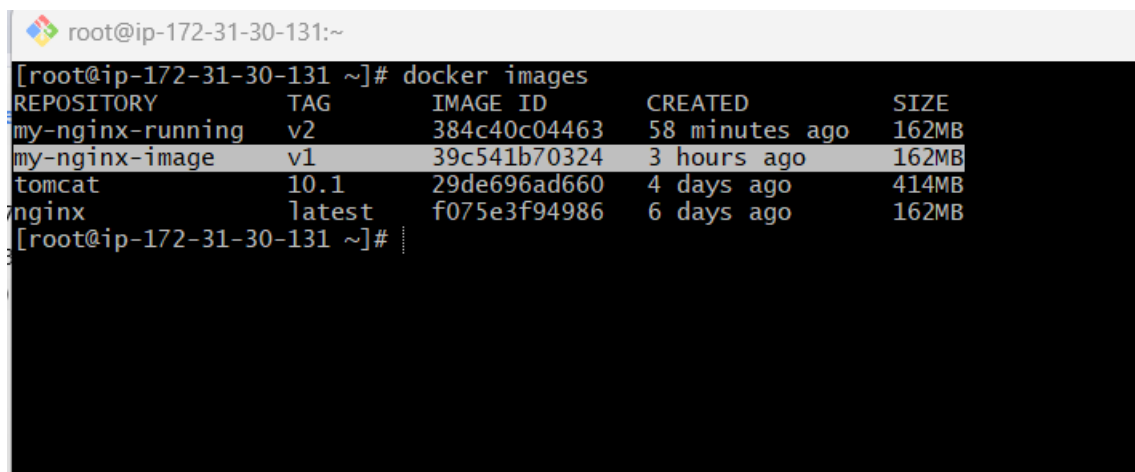
docker images

Example:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-nginx-image	v1	abc123456789	10 minutes ago	162MB
nginx	latest	def987654321	2 hours ago	162MB

Assume the image we want to transfer is:

my-nginx-image:v1



```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# docker images  
REPOSITORY          TAG             IMAGE ID         CREATED          SIZE  
my-nginx-running    v2              384c40c04463    58 minutes ago  162MB  
my-nginx-image      v1              39c541b70324    3 hours ago     162MB  
tomcat              10.1            29de696ad660    4 days ago      414MB  
nginx               latest          f075e3f94986    6 days ago      162MB  
[root@ip-172-31-30-131 ~]#
```

3. Save the Docker Image as a TAR File

Use the docker save command:

docker save -o my-nginx-image.tar my-nginx-image:v1

Explanation

- docker save → Saves a Docker image into a file.
- -o → Specifies the output file.
- my-nginx-image.tar → Name of the image archive.
- my-nginx-image:v1 → Docker image that we want to save.

4. Verify the TAR File

Check whether the file was created:

ls -lh my-nginx-image.tar

Example:

```
-rw-r--r-- 1 user user 162M Aug 26 20:00 my-nginx-image.tar
```

You can also verify the file type:

file my-nginx-image.tar

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# docker images  
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE  
my-nginx-running    v2              384c40c04463   58 minutes ago 162MB  
my-nginx-image      v1              39c541b70324   3 hours ago    162MB  
tomcat               10.1            29de696ad660   4 days ago     414MB  
nginx                latest          f075e3f94986   6 days ago     162MB  
[root@ip-172-31-30-131 ~]# docker save -o my-nginx-image.tar my-nginx-image:v1  
[root@ip-172-31-30-131 ~]# ls -lh  
total 158M  
drwxr-xr-x. 2 root root  42 Aug 26 12:01 my-nginx  
-rw----- 1 root root 158M Aug 26 15:18 my-nginx-image.tar  
drwxr-xr-x. 2 root root  42 Aug 25 06:54 nginx-docker  
drwxr-xr-x. 3 root root  62 Aug 24 16:12 tomcat-app  
[root@ip-172-31-30-131 ~]#
```

5. Copy the TAR File from Local machine to the Docker Server

If the Docker server is an AWS EC2 instance, use scp.

Syntax

```
scp -i <key.pem> <image.tar> <username>@<server-ip>:<destination>
```

```
scp -i server-1.pem my-nginx-image.tar ec2-user@13.221.26.102:/home/ec2-user/
```

Example

For an Amazon Linux EC2 server:

```
scp -i mykey.pem my-nginx-image.tar ec2-user@<EC2-PUBLIC-IP>:/home/ec2-user/
```

```
scp -i /c/Users/braj/Downloads/server-1.pem Local-nginx.tar ec2-user@13.221.26.102:/tmp
```

6. SSH into the Docker Server

Connect to the remote Docker server:

```
ssh -i mykey.pem ec2-user@<EC2-PUBLIC-IP>
```

```
MINGW64:/c/Users/brajk/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ pwd
/c/Users/brajk/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ scp -i /c/Users/brajk/Downloads/server-1.pem Local-nginx.tar ec2-user@13.221.26.102:/tmp
Local-nginx.tar                               100%  63MB  2.0MB/s  00:32
brajk@RajKumari MINGW64 ~/Downloads
$ |
```

7. Verify the TAR File on the Docker Server

After logging into the server:

```
ls -lh
```

```
cd /tmp
```

```
root@ip-172-31-30-131:/tmp
[root@ip-172-31-30-131 ~]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
my-nginx-running    v2           384c40c04463     About an hour ago 162MB
my-nginx-image      v1           39c541b70324     4 hours ago      162MB
[root@ip-172-31-30-131 ~]# cd tmp
-bash: cd: tmp: No such file or directory
[root@ip-172-31-30-131 ~]# cd /tmp
[root@ip-172-31-30-131 tmp]# ls
Local-nginx.tar
hsperfdata_root
systemd-private-e75c6e32cc6e42119ca1f481c686e0b1-chrond.service-vTqwD0
systemd-private-e75c6e32cc6e42119ca1f481c686e0b1-dbus-broker.service-k9HREP
systemd-private-e75c6e32cc6e42119ca1f481c686e0b1-policy-routes@ens5.service-HxCn0d
systemd-private-e75c6e32cc6e42119ca1f481c686e0b1-systemd-logind.service-3Ffdi9
systemd-private-e75c6e32cc6e42119ca1f481c686e0b1-systemd-resolved.service-MKObxW
[root@ip-172-31-30-131 tmp]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
my-nginx-running    v2           384c40c04463     2 hours ago      162MB
my-nginx-image      v1           39c541b70324     4 hours ago      162MB
[root@ip-172-31-30-131 tmp]# |
```

8. Load the Image into Docker

Now load the image using:

```
docker load -i Local-nginx.tar
```

You should see output similar to:

Loaded image: Local-nginx:v1

This means the image was successfully loaded into Docker.

9. Verify the Image

Run:

docker images

You should see:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-nginx-image	v1	abc123456789	10 minute	

```
root@ip-172-31-30-131:/tmp
[root@ip-172-31-30-131 tmp]# ls
Local-nginx.tar
hsperfdata_root
systemd-private-e75c6e32cc6e42119ca1f481c686e0b1-chrond.service-vTqW0
systemd-private-e75c6e32cc6e42119ca1f481c686e0b1-dbus-broker.service-k9HREP
systemd-private-e75c6e32cc6e42119ca1f481c686e0b1-policy-routes@ens5.service-HxCn0d
systemd-private-e75c6e32cc6e42119ca1f481c686e0b1-systemd-logind.service-3FfDi9
systemd-private-e75c6e32cc6e42119ca1f481c686e0b1-systemd-resolved.service-MK0bXW
[root@ip-172-31-30-131 tmp]# docker load -i Local-nginx.tar
411a86676185: Loading layer 29.79MB/29.79MB
070ef859f070: Loading layer 33.53MB/33.53MB
e8fb5de3ef79: Loading layer 628B/628B
f109061e6486: Loading layer 954B/954B
f29840e7d6e5: Loading layer 404B/404B
b0020123c41b: Loading layer 1.21kB/1.21kB
a8fae4102358: Loading layer 1.395kB/1.395kB
Loaded image: nginx:latest
[root@ip-172-31-30-131 tmp]# docker images
REPOSITORY          TAG          IMAGE ID        CREATED         SIZE
my-nginx-running    v2           384c40c04463    2 hours ago    162MB
my-nginx-image      v1           39c541b70324    4 hours ago    162MB
nginx               latest       38bec41bfa      40 hours ago    162MB
[root@ip-172-31-30-131 tmp]#
```

10. Run a Container from the Loaded Image

Now create a container:

docker run -d --name my-nginx-container -p 80:80 my-nginx-image:v1

Check the running container:

docker ps

Example:

CONTAINER ID	IMAGE	PORTS	NAMES
xxxxxxxxxxxx	my-nginx-image:v1	0.0.0.0:80->80/tcp	

```
root@ip-172-31-30-131:/tmp
[root@ip-172-31-30-131 tmp]# docker container run -itd -p 80:80 nginx:latest
f9a85d62d33e143e93e139432b96f7d0402e03511bf0ad2bfe08d5feb319fc36
[root@ip-172-31-30-131 tmp]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
f9a85d62d33e   nginx:latest   "/docker-entrypoint..." 36 seconds ago Up 36 seconds 0.0.0.0:80->80/tcp
::80->80/tcp   pensive_pascal
[root@ip-172-31-30-131 tmp]#
```

11. Test the Application

From the Docker server:

curl http://localhost



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

3. Create Docker image using alpine and customize with tomcat.

1. Objective

Create a custom Docker image based on **Alpine Linux**, install **Java**, download and configure **Apache Tomcat**, expose Tomcat port 8080, and start Tomcat when the container launches.

The architecture is:

Alpine Linux



Install Java



Download Tomcat



Configure Tomcat



Expose Port 8080



Start Tomcat



Custom Tomcat Docker Image

2. Create a Directory

Create a working directory on the Docker server:

```
mkdir alpine-tomcat
```

```
cd alpine-tomcat
```

Verify: `pwd`

```
root@ip-172-31-30-131:~/alpine-tomcat
[root@ip-172-31-30-131 ~]# mkdir alpine-tomcat
[root@ip-172-31-30-131 ~]# cd alpine-tomcat/
[root@ip-172-31-30-131 alpine-tomcat]# ls
[root@ip-172-31-30-131 alpine-tomcat]# pwd
/root/alpine-tomcat
[root@ip-172-31-30-131 alpine-tomcat]#
```

3. Create the Dockerfile

Create a file named Dockerfile:

```
vi Dockerfile
```

Add the following content:

```
FROM alpine:latest
```

LABEL maintainer="Rajkumari"

RUN apk update && \

apk add --no-cache openjdk17-jre wget tar bash

ENV CATALINA_HOME=/opt/tomcat

ENV PATH=\$CATALINA_HOME/bin:\$PATH

RUN wget -q https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.49/bin/apache-tomcat-10.1.49.tar.gz && \

mkdir -p /opt && \

tar -xzf apache-tomcat-10.1.49.tar.gz -C /opt && \

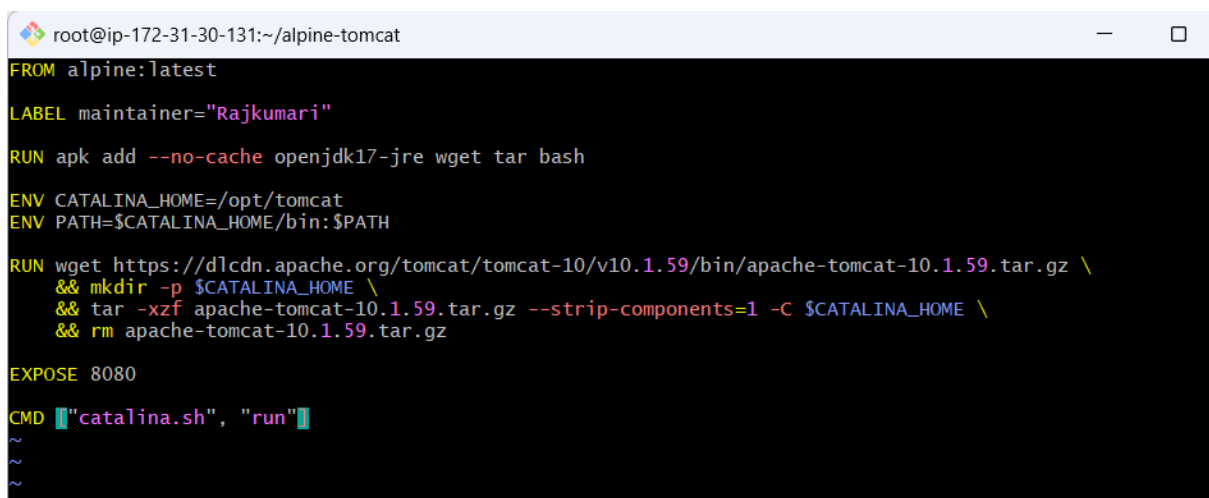
mv /opt/apache-tomcat-10.1.49 /opt/tomcat && \

rm apache-tomcat-10.1.49.tar.gz

EXPOSE 8080

CMD ["catalina.sh", "run"]

Note: Tomcat versions change over time. If that exact version is no longer available, replace the download URL and directory name with the current Tomcat 10.1 version.

A screenshot of a Dockerfile for Alpine Tomcat. The file is named 'alpine-tomcat' and is located at the path '~/.alpine-tomcat'. The Dockerfile contains the following instructions: FROM alpine:latest, LABEL maintainer="Rajkumari", RUN apk add --no-cache openjdk17-jre wget tar bash, ENV CATALINA_HOME=/opt/tomcat, ENV PATH=\$CATALINA_HOME/bin:\$PATH, RUN wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.59/bin/apache-tomcat-10.1.59.tar.gz \&& mkdir -p \$CATALINA_HOME \&& tar -xzf apache-tomcat-10.1.59.tar.gz --strip-components=1 -C \$CATALINA_HOME \&& rm apache-tomcat-10.1.59.tar.gz, EXPOSE 8080, and CMD ["catalina.sh", "run"]. The screenshot shows the Dockerfile content in a terminal window with a dark background and light-colored text. The window title is 'root@ip-172-31-30-131:~/alpine-tomcat'. The Dockerfile content is as follows:

```
FROM alpine:latest
LABEL maintainer="Rajkumari"
RUN apk add --no-cache openjdk17-jre wget tar bash
ENV CATALINA_HOME=/opt/tomcat
ENV PATH=$CATALINA_HOME/bin:$PATH
RUN wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.59/bin/apache-tomcat-10.1.59.tar.gz \
&& mkdir -p $CATALINA_HOME \
&& tar -xzf apache-tomcat-10.1.59.tar.gz --strip-components=1 -C $CATALINA_HOME \
&& rm apache-tomcat-10.1.59.tar.gz
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

4. Understand the Dockerfile

FROM

```
FROM alpine:latest
```

This uses the latest Alpine Linux image as the base image.

Alpine is a very small Linux distribution, which helps keep the Docker image relatively lightweight.

LABEL

```
LABEL maintainer="Rajkumari"
```

Adds metadata to the Docker image.

You can replace Rajkumari with your name.

Install Java and Required Packages

```
RUN apk update && \
```

```
    apk add --no-cache openjdk17-jre wget tar bash
```

Here:

- apk → Alpine Linux package manager
- openjdk17-jre → Java Runtime Environment
- wget → Downloads Tomcat
- tar → Extracts the Tomcat archive
- bash → Provides Bash shell support

Tomcat requires Java to run.

5. Set Tomcat Environment Variables

```
ENV CATALINA_HOME=/opt/tomcat
```

```
ENV PATH=$CATALINA_HOME/bin:$PATH
```

CATALINA_HOME defines the location where Tomcat is installed.

The PATH modification allows commands such as:

```
catalina.sh
```

to be executed directly.

6. Download and Install Tomcat

```
RUN wget -q https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.49/bin/apache-  
tomcat-10.1.49.tar.gz && \
```

```
mkdir -p /opt && \
```

```
tar -xzf apache-tomcat-10.1.49.tar.gz -C /opt && \
```

```
mv /opt/apache-tomcat-10.1.49 /opt/tomcat && \
```

```
rm apache-tomcat-10.1.49.tar.gz
```

This performs several operations:

1. Downloads Tomcat.
2. Creates /opt.
3. Extracts the Tomcat archive.
4. Renames the extracted directory to /opt/tomcat.
5. Removes the downloaded .tar.gz file.

After installation, Tomcat will be located at:

```
/opt/tomcat
```

7. Expose Tomcat Port

EXPOSE 8080

This documents that the application inside the container listens on port 8080.

It does **not** publish the port to the host by itself.

The port is published when using `docker run -p`.

8. Start Tomcat

CMD ["catalina.sh", "run"]

This starts Tomcat in the foreground.

Keeping Tomcat in the foreground is important because Docker expects the main application process to remain running.

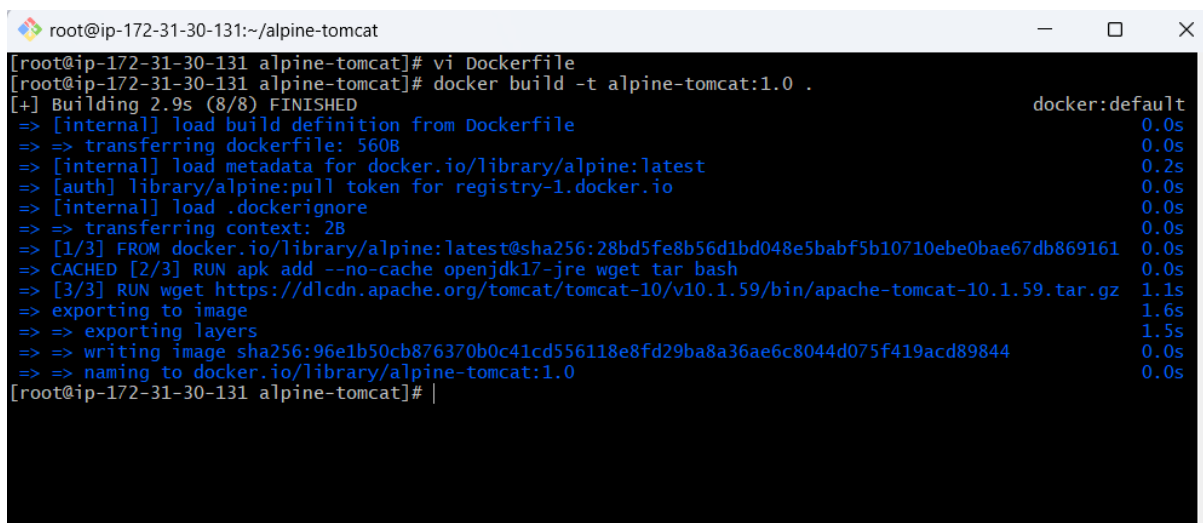
. Build the Docker Image

From the directory containing the Dockerfile:

`docker build -t alpine-tomcat:1.0 .`

Explanation:

- `docker build` → Builds an image.
- `-t alpine-tomcat:1.0` → Gives the image a name and tag.
- `.` → Uses the current directory as the build context.

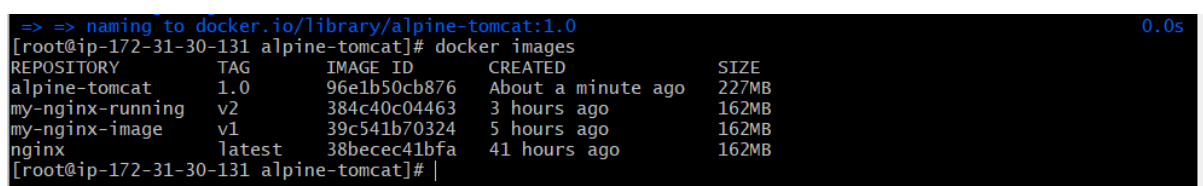


```
root@ip-172-31-30-131:~/alpine-tomcat
[root@ip-172-31-30-131 alpine-tomcat]# vi Dockerfile
[root@ip-172-31-30-131 alpine-tomcat]# docker build -t alpine-tomcat:1.0 .
[+] Building 2.9s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 560B                             0.0s
=> [internal] load metadata for docker.io/library/alpine:latest 0.2s
=> [auth] library/alpine:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                 0.0s
=> => transferring context: 2B                                    0.0s
=> [1/3] FROM docker.io/library/alpine:latest@sha256:28bd5fe8b56d1bd048e5babf5b10710ebe0bae67db869161 0.0s
=> CACHED [2/3] RUN apk add --no-cache openjdk17-jre wget tar bash 0.0s
=> [3/3] RUN wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.59/bin/apache-tomcat-10.1.59.tar.gz 1.1s
=> exporting to image                                           1.6s
=> => exporting layers                                           1.5s
=> => writing image sha256:96e1b50cb876370b0c41cd556118e8fd29ba8a36ae6c8044d075f419acd89844 0.0s
=> => naming to docker.io/library/alpine-tomcat:1.0            0.0s
[root@ip-172-31-30-131 alpine-tomcat]#
```

10. Verify the Image

Run:

`docker images`



```
=> => naming to docker.io/library/alpine-tomcat:1.0            0.0s
[root@ip-172-31-30-131 alpine-tomcat]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
alpine-tomcat       1.0         96e1b50cb876 About a minute ago 227MB
my-nginx-running    v2          384c40c04463 3 hours ago    162MB
my-nginx-image      v1          39c541b70324 5 hours ago    162MB
nginx               latest      38becec41bfa 41 hours ago    162MB
[root@ip-172-31-30-131 alpine-tomcat]#
```

11. Run the Tomcat Container

Run:

```
docker run -d \
  --name alpine-tomcat-container \
  -p 8080:8080 \
  alpine-tomcat:1.0
```

Explanation:

-p 8080:8080

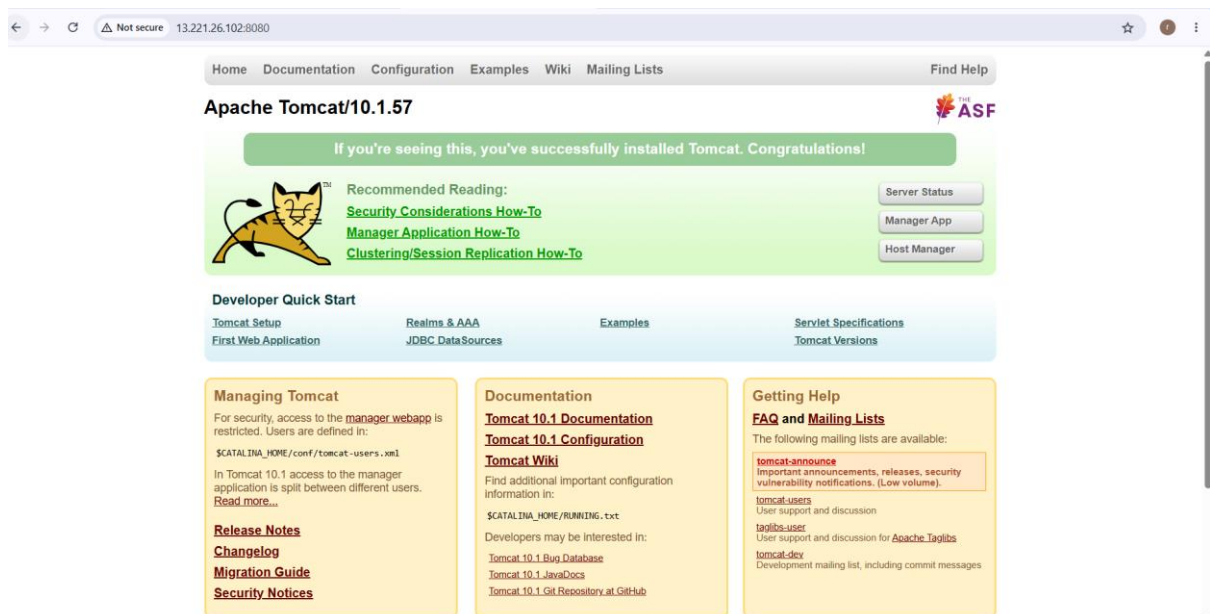
means:

Host Port 8080 → Container Port 8080

12. Test Tomcat

From the Docker server:

```
curl http://localhost:8080
```



4. Create single stage and multi stage docker file using this

source code: <https://github.com/betawins/multi-stage-example.git>

Single-stage Dockerfile — Maven build and runtime are in the same image.

Multi-stage Dockerfile — Maven/JDK are used only for building; the final image contains only what is required to run the application. This is the main benefit of multi-stage builds.

1. Source Code

GitHub repository:

[betawins/multi-stage-example](https://github.com/betawins/multi-stage-example)

The repository contains:

.mvn/

src/

mvnw

mvnw.cmd

pom.xml

Dockerfile

README.md

The application is a Spring Boot application and uses Maven for building.

Part 1: Clone the Source Code

2. Clone the Repository

On your Docker server:

```
git clone https://github.com/betawins/multi-stage-example.git
```

Go inside the project:

```
cd multi-stage-example
```

Check the files:

```
ls -la
```

You should see:

```
.mvn
```

```
src
```

```
mvnw
```

```
mvnw.cmd
```

```
pom.xml
```

```
Dockerfile
```

```
README.md
```

```
root@ip-172-31-30-131:~/multi-stage-example
[root@ip-172-31-30-131 ~]# git --version
git version 2.50.1
[root@ip-172-31-30-131 ~]# git clone https://github.com/betawins/multi-stage-example.git
Cloning into 'multi-stage-example'...
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 31 (delta 2), reused 1 (delta 1), pack-reused 24 (from 1)
Receiving objects: 100% (31/31), 53.25 KiB | 26.63 MiB/s, done.
Resolving deltas: 100% (3/3), done.
[root@ip-172-31-30-131 ~]# ls -la
total 161664
dr-xr-x---.  9 root root    16384 Aug 26 17:09 .
dr-xr-xr-x. 18 root root     237 Aug  1 04:12 ..
-rw-----.  1 root root    9309 Aug 26 15:24 .bash_history
-rw-r--r--.  1 root root     18 Feb  2 2023 .bash_logout
-rw-r--r--.  1 root root    141 Feb  2 2023 .bash_profile
-rw-r--r--.  1 root root    429 Feb  2 2023 .bashrc
-rw-r--r--.  1 root root    100 Feb  2 2023 .cshrc
drwx-----.  3 root root     82 Aug 25 09:38 .docker
-rw-----.  1 root root     20 Aug 25 09:35 .lessht
drwx-----.  2 root root     48 Aug 26 15:23 .ssh
-rw-r--r--.  1 root root    129 Feb  2 2023 .tcshrc
-rw-----.  1 root root   12989 Aug 26 16:54 .viminfo

[root@ip-172-31-30-131 ~]# cd multi-stage-example/
[root@ip-172-31-30-131 multi-stage-example]# ls
Dockerfile README.md mvnw mvnw.cmd pom.xml src
[root@ip-172-31-30-131 multi-stage-example]#
```

Part 2: Single-Stage Dockerfile

3. What is a Single-Stage Build?

In a single-stage Dockerfile, the same image contains:

Maven

JDK

Source Code

Dependencies

Build Tools

Application JAR

Runtime

Therefore, the final image can become larger because build-time tools and files remain in the image.

4. Create Single-Stage Dockerfile

Create:

```
vi Dockerfile.single
```

Add:

```
FROM maven:3.8.6-eclipse-temurin-8
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN ./mvnw clean package -DskipTests
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["java", "-jar", "/app/target/multi-stage-example-0.0.1-SNAPSHOT.jar"]
```

```
root@ip-172-31-30-131:~/multi-stage-example
FROM maven:3.8.6-eclipse-temurin-8
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/target/multi-stage-example-0.0.1-SNAPSHOT.jar"]
~
~
~
~
```

5. Explanation of Single-Stage Dockerfile

FROM

FROM maven:3.8.6-eclipse-temurin-8

This image provides:

- Maven
- JDK 8
- Java build environment

The application in the repository uses Spring Boot 2.2.6.RELEASE and Java 8 in its Maven configuration.

WORKDIR

WORKDIR /app

Creates /app as the working directory.

All subsequent commands execute from this directory.

COPY

COPY . .

Copies the complete project into /app.

For example:

/app

└── src

└── pom.xml

└── mvnw

└── .mvn

Build the Application

RUN `./mvnw clean package -DskipTests`

This executes Maven and creates the JAR file under:

`/app/target/`

The `-DskipTests` option skips test execution.

If you want to run tests during the image build, use:

RUN `./mvnw clean package`

EXPOSE

EXPOSE 8080

The Spring Boot application listens on port 8080.

ENTRYPOINT

ENTRYPOINT ["java", "-jar", "/app/target/multi-stage-example-0.0.1-SNAPSHOT.jar"]

This starts the Spring Boot application when the container starts.

6. Build the Single-Stage Image

Run:

`docker build -f Dockerfile.single -t multi-stage-single:v1 .`

Explanation:

-f Dockerfile.single

specifies the Dockerfile to use.

-t multi-stage-single:v1

assigns the image name and tag.

.

specifies the current directory as the build context.

```
root@ip-172-31-30-131:~/multi-stage-example
[root@ip-172-31-30-131 multi-stage-example]# vi Dockerfile.single
[root@ip-172-31-30-131 multi-stage-example]# docker build -f Dockerfile.single -t multi-stage-single:v1 .
[+] Building 30.8s (10/10) FINISHED
=> [internal] load build definition from Dockerfile.single                                0.0s
=> => transferring dockerfile: 296B                                                    0.0s
=> [internal] load metadata for docker.io/library/maven:3.8.6-eclipse-temurin-8        0.3s
=> [auth] library/maven:pull token for registry-1.docker.io                          0.0s
=> [internal] load .dockerignore                                                       0.0s
=> => transferring context: 2B                                                         0.0s
=> [internal] load build context                                                       0.0s
=> => transferring context: 7.61kB                                                    0.0s
=> [1/4] FROM docker.io/library/maven:3.8.6-eclipse-temurin-8@sha256:9e0cb7a6d15f3b0b59e11db52da2ec06 0.0s
=> CACHED [2/4] WORKDIR /app                                                         0.0s
=> [3/4] COPY . .                                                                    0.1s
=> [4/4] RUN mvn clean package -DskipTests                                           29.2s
=> exporting to image                                                                1.1s
=> => exporting layers                                                                1.0s
=> => writing image sha256:b106e3f4c768f76980907eb46d47615b9df4846d897e538c59fd7e99fa2e43d4 0.0s
=> => naming to docker.io/library/multi-stage-single:v1                             0.0s
[root@ip-172-31-30-131 multi-stage-example]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
multi-stage-single   v1          b106e3f4c768 41 seconds ago 469MB
alpine-tomcat        1.0         96e1b50cb876 28 minutes ago 227MB
my-nginx-running     v2          384c40c04463 3 hours ago    162MB
```

7. Verify the Single-Stage Image

docker images

You should see:

REPOSITORY	TAG	IMAGE ID	SIZE
------------	-----	----------	------

multi-stage-single	v1	xxxxxxxxxxxxx	...
--------------------	----	---------------	-----

8. Run the Single-Stage Container

docker run -d \

--name multi-stage-single-container \

-p 8080:8080 \

multi-stage-single:v1

9. Verify the Container

`docker ps`

Expected:

CONTAINER ID	IMAGE	PORTS	NAMES
xxxxxxxxxxxxx	multi-stage-single:v1	0.0.0.0:8080->808	

```
root@ip-172-31-30-131:~/multi-stage-example
[root@ip-172-31-30-131 multi-stage-example]# docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
multi-stage-single  v1             b106e3f4c768   5 minutes ago  469MB
alpine-tomcat       1.0           96e1b50cb876   33 minutes ago 227MB
my-nginx-running    v2            384c40c04463   3 hours ago    162MB
my-nginx-image      v1            39c541b70324   5 hours ago    162MB
nginx               latest        38becec41bfa   41 hours ago   162MB
[root@ip-172-31-30-131 multi-stage-example]# docker run -d --name multi-stage-single-container -p 8083:8080 m
ulti-stage-single:v1
b21290841b390e474652de947cc10de170c23793bbb0d5e7625bc7a9b8c447d3
[root@ip-172-31-30-131 multi-stage-example]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
b21290841b39   multi-stage-single:v1   "java -jar /app/targ..." 7 seconds ago  Up 5 seconds  0.0.0.
0:8083->8080/tcp, :::8083->8080/tcp
multi-stage-single-container
f9a85d62d33e   nginx:latest   "/docker-entrypoint..." About an hour ago  Up About an hour  0.0.0.
0:80->80/tcp, :::80->80/tcp
pensive_pascal
[root@ip-172-31-30-131 multi-stage-example]#
```

10. Check Application Logs

`docker logs multi-stage-single-container`

You should see Spring Boot startup messages.

11. Test the Application

From the Docker server:

`curl http://localhost:8080`

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Wed Aug 26 17:31:39 UTC 2026

There was an unexpected error (type=Not Found, status=404).

No message available

Part 3: Multi-Stage Dockerfile

12. What is a Multi-Stage Build?

A multi-stage Dockerfile uses multiple FROM instructions.

The first stage is responsible for building the application:

Source Code

↓

Maven

↓

JDK

↓

JAR

The second stage is responsible only for running the application:

JRE

+

JAR

↓

Final Image

Docker supports copying only the required build artifact from one stage to another using:

`COPY --from=builder`

This keeps build tools out of the final image.

```
root@ip-172-31-30-131:~/multi-stage-example
# Stage 1: Build
FROM maven:3.8.6-eclipse-temurin-8 AS builder

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

# Stage 2: Runtime
FROM eclipse-temurin:8-jre

WORKDIR /app

COPY --from=builder /app/target/multi-stage-example-0.0.1-SNAPSHOT.jar /app/app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/app.jar"]
~
~
~
-- INSERT --
```

14. Understand the Multi-Stage Dockerfile

Stage 1 — Builder

FROM maven:3.8.6-eclipse-temurin-8 AS builder

This stage contains:

Maven

JDK

Source Code

Dependencies

Build Tools

The name:

builder

allows us to refer to this stage later.

Copy Source Code

WORKDIR /app

COPY . .

The application source code is copied into:

/app

Build the Application

RUN `./mvnw clean package -DskipTests`

This creates:

/app/target/multi-stage-example-0.0.1-SNAPSHOT.jar

15. Stage 2 — Runtime

FROM eclipse-temurin:8-jre

A JRE-only image is used for runtime instead of the Maven/JDK build environment.

The final image therefore doesn't need Maven or the source code.

16. Copy Only the JAR

This is the most important line:

COPY --from=builder /app/target/multi-stage-example-0.0.1-SNAPSHOT.jar app.jar

It means:

builder stage

|

| JAR only

↓

final stage

The Maven installation, source code, and build-time files are not copied into the final image.

This is the key advantage of multi-stage builds.

17. Expose Application Port

EXPOSE 8080

The Spring Boot application listens on port 8080.

18. Start the Application

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

This starts the Spring Boot application.

19. Build the Multi-Stage Image

Run:

```
docker build -f Dockerfile.multi -t multi-stage-multi:v1 .
```

Docker will execute:

Stage 1

↓

Maven Build

↓

JAR Created

↓

Stage 2

↓

Copy JAR

↓

Final Runtime Image

20. Verify the Image

docker images

You should see:

REPOSITORY	TAG	IMAGE ID	SIZE
------------	-----	----------	------

multi-stage-multi v1 xxxxxxxxxxxx ...

multi-stage-single v1 xxxxxxxxxxxx ...

The multi-stage image should generally be smaller because build-time dependencies aren't included in the final image.

```
root@ip-172-31-30-131:~/multi-stage-example
=> [internal] load build definition from Dockerfile.multi 0.0s
=> => transferring dockerfile: 432B 0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:8-jre 0.1s
=> [internal] load metadata for docker.io/library/maven:3.8.6-eclipse-temurin-8 0.1s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:8-jre@sha256:e8950bdf2ba05642262f62197910032 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 7.84kB 0.0s
=> [builder 1/4] FROM docker.io/library/maven:3.8.6-eclipse-temurin-8@sha256:9e0cb7a6d15f3b0b59e11db5 0.0s
=> CACHED [stage-1 2/3] WORKDIR /app 0.0s
=> CACHED [builder 2/4] WORKDIR /app 0.0s
=> [builder 3/4] COPY . . 0.1s
=> [builder 4/4] RUN mvn clean package -DskipTests 30.0s
=> [stage-1 3/3] COPY --from=builder /app/target/multi-stage-example-0.0.1-SNAPSHOT.jar /app/app.jar 0.1s
=> exporting to image 0.2s
=> => exporting layers 0.1s
=> => writing image sha256:f1941c3ca17cf99afb901b716f866114ac31e2c38643dd8e3efd51784561a5a1 0.0s
=> => naming to docker.io/library/multi-stage-multi:v1 0.0s
[root@ip-172-31-30-131 multi-stage-example]# docker images
REPOSITORY          TAG       IMAGE ID       CREATED        SIZE
multi-stage-multi    v1        f1941c3ca17c   2 minutes ago  282MB
multi-stage-single   v1        b106e3f4c768   19 minutes ago 469MB
alpine-tomcat        1.0       96e1b50cb876   47 minutes ago 227MB
```

21. Run the Multi-Stage Container

docker run -d \

--name multi-stage-multi-container \

-p 8081:8080 \

multi-stage-multi:v1

If the single-stage container is already using port 8080, stop and remove it first:

docker stop multi-stage-single-container

docker rm multi-stage-single-container

Then run the multi-stage container.

22. Verify the Container

docker ps

Example:

CONTAINER ID	IMAGE	PORTS	NAMES
--------------	-------	-------	-------

xxxxxxxxxxx	multi-stage-multi:v1	0.0.0.0:8080->8080/tcp	multi-stage-multi-container
-------------	----------------------	------------------------	-----------------------------

```
root@ip-172-31-30-131:~/multi-stage-example
[root@ip-172-31-30-131 multi-stage-example]# docker run -d --name multi-stage-multi-container -p 8081:8080 multi-stage-multi:v1
e7018a627281fc345aae01adea44b6559fd44514e8845791a5937cacaf8640d5
[root@ip-172-31-30-131 multi-stage-example]# docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS        PORTS
e7018a627281   multi-stage-multi:v1               "java -jar /app/app...."              6 seconds ago  Up 5 seconds  0.0.0.0
:8081->8080/tcp, :::8081->8080/tcp
f9a85d62d33e   nginx:latest                       "/docker-entrypoint...."              About an hour ago  Up About an hour  0.0.0.0
:80->80/tcp, :::80->80/tcp
pensive_pascal
[root@ip-172-31-30-131 multi-stage-example]#
```

23. Check Logs

`docker logs multi-stage-multi-container`

24. Test the Application

`curl http://localhost:8080`



Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Wed Aug 26 17:48:42 UTC 2026

There was an unexpected error (type=Not Found, status=404).

No message available

5. Install docker compose and execute sample application.

1. Objective

The objective is to:

1. Install Docker Compose on the Docker server.
2. Verify the Docker Compose installation.
3. Create a sample multi-container application.
4. Create a compose.yaml file.
5. Build and start the application using Docker Compose.

6. Verify the running containers.
7. Check application logs.
8. Access the application.
9. Stop and remove the application.

Docker Compose allows us to define multiple services in a YAML file and manage the complete application stack using commands such as `docker compose up`, `docker compose ps`, and `docker compose down`.

Part 1: Verify Docker Installation

Before installing Compose, verify that Docker is already installed.

`docker --version`

Example:

Docker version 28.x.x

Check that Docker is running:

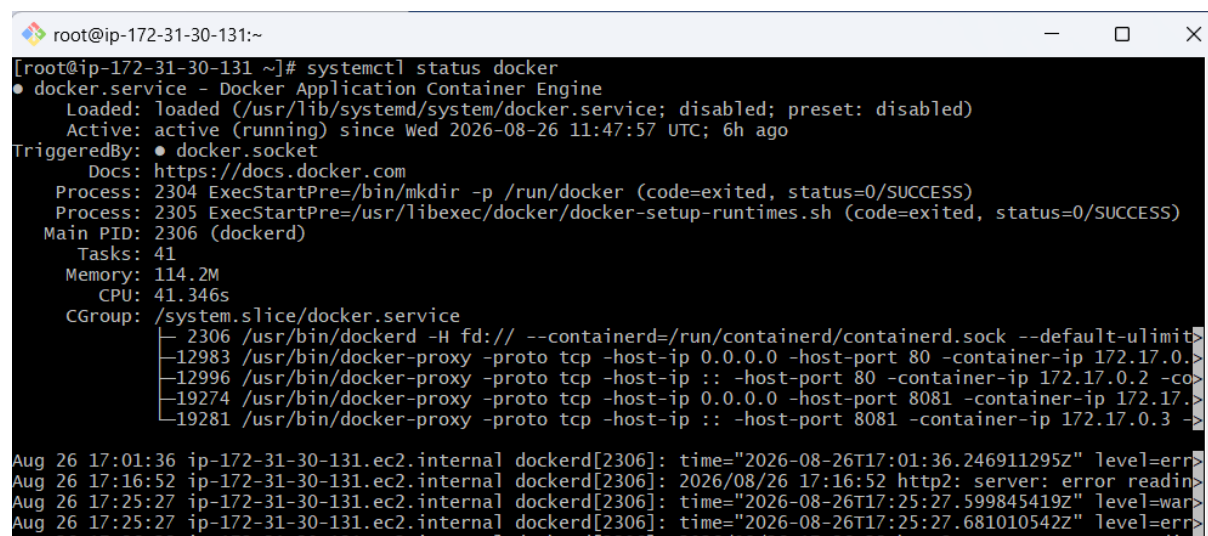
`sudo systemctl status docker`

If Docker is not running:

`sudo systemctl start docker`

Enable Docker at boot:

`sudo systemctl enable docker`



```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# systemctl status docker  
● docker.service - Docker Application Container Engine  
   Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; preset: disabled)  
   Active: active (running) since Wed 2026-08-26 11:47:57 UTC; 6h ago  
 TriggeredBy: ● docker.socket  
    Docs: https://docs.docker.com  
   Process: 2304 ExecStartPre=/bin/mkdir -p /run/docker (code=exited, status=0/SUCCESS)  
   Process: 2305 ExecStartPre=/usr/libexec/docker/docker-setup-runtimes.sh (code=exited, status=0/SUCCESS)  
 Main PID: 2306 (dockerd)  
    Tasks: 41  
   Memory: 114.2M  
      CPU: 41.346s  
   CGroup: /system.slice/docker.service  
           └─ 2306 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit>  
             12983 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 80 -container-ip 172.17.0.2 -co>  
             12996 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 80 -container-ip 172.17.0.2 -co>  
             19274 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8081 -container-ip 172.17.0.2 -co>  
             19281 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8081 -container-ip 172.17.0.3 -co>  
Aug 26 17:01:36 ip-172-31-30-131.ec2.internal dockerd[2306]: time="2026-08-26T17:01:36.246911295Z" level=err>  
Aug 26 17:16:52 ip-172-31-30-131.ec2.internal dockerd[2306]: 2026/08/26 17:16:52 http2: server: error readin>  
Aug 26 17:25:27 ip-172-31-30-131.ec2.internal dockerd[2306]: time="2026-08-26T17:25:27.599845419Z" level=war>  
Aug 26 17:25:27 ip-172-31-30-131.ec2.internal dockerd[2306]: time="2026-08-26T17:25:27.681010542Z" level=err>  
Aug 26 17:26:22 ip-172-31-30-131.ec2.internal dockerd[2306]: 2026/08/26 17:26:22 http2: server: error readin>
```

Part 2: Install Docker Compose

2. Install Docker Compose Plugin

For an RPM-based Linux system such as Amazon Linux/RHEL/CentOS, Docker's repository-based installation uses:

```
sudo yum update
```

```
sudo yum install docker-compose-plugin
```

Docker's official documentation recommends installing the Compose plugin through the Docker repository when possible because it is easier to maintain and update.

3. Verify Docker Compose

Run:

```
docker compose version
```

Example:

```
Docker Compose version v5.x.x
```

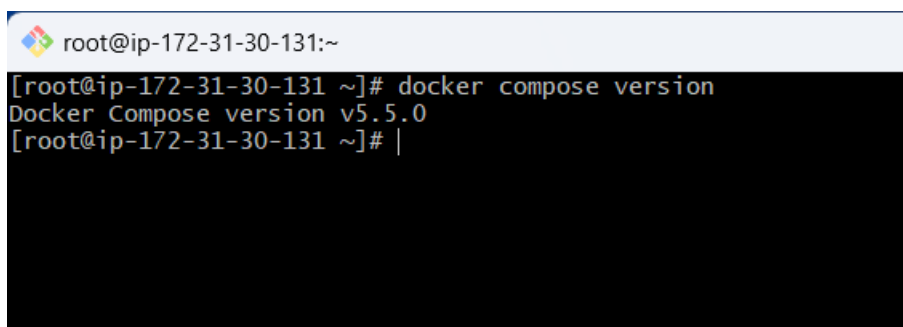
The important point is that the command is:

```
docker compose
```

not:

```
docker-compose
```

The latter refers to the older standalone/legacy installation.



```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# docker compose version  
Docker Compose version v5.5.0  
[root@ip-172-31-30-131 ~]# |
```

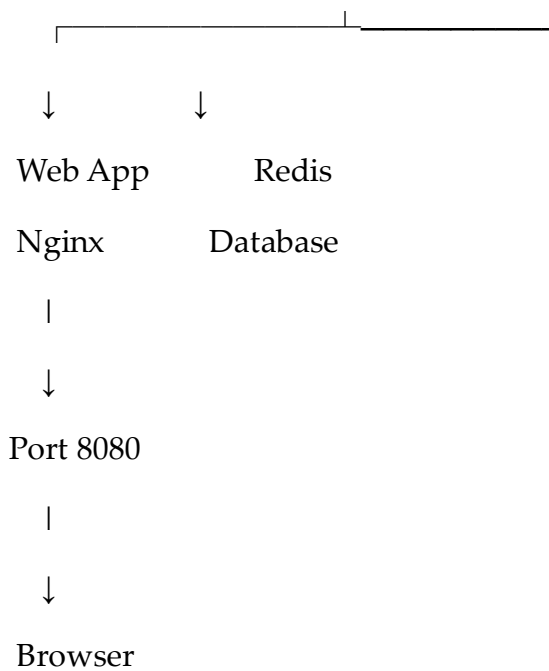
Part 3: Create a Sample Application

For this demonstration, we will create a simple **Nginx + Redis** application.

The architecture will be:

Docker Compose

|



4. Create a Project Directory

`mkdir compose-demo`

`cd compose-demo`

Verify:

`pwd`

5. Create an HTML Application

Create an HTML file:

`vi index.html`

```
root@ip-172-31-30-131:~/compose-demo
<!DOCTYPE html>
<html>
<head>
  <title>Docker Compose Demo</title>
</head>
<body>
  <h1>Hello Rajkumari from Docker Compose!</h1>
  <p>This application is running using Docker Compose.</p>
</body>
</html>
```

6. Create a Dockerfile

Create:

vi Dockerfile

```
root@ip-172-31-30-131:~/compose-demo
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
~
~
~
~
~
~
```

This creates a custom Nginx image containing our HTML application.

7. Create the Compose File

Create:

vi compose.yaml

```
root@ip-172-31-30-131:~/compose-demo
services:
  web:
    build: .
    container_name: compose-web
    ports:
      - "8080:80"

  redis:
    image: redis:alpine
    container_name: compose-redis
~
~
~
~
~
~
```

8. Understand the Compose File

services

services:

Defines the containers/services that belong to our application.

Web Service

web:

This defines our web application service.

Build

build: .

Docker Compose builds the image using the Dockerfile in the current directory.

Container Name

container_name: compose-web

Assigns a specific name to the container.

Port Mapping

ports:

- "8080:80"

This means:

Host Port 8080

↓

Container Port 80

Therefore, Nginx listens internally on port 80, while we access it from the host using port 8080.

9. Redis Service

redis:

image: redis:alpine

Instead of building an image, Compose pulls the Redis Alpine image from the registry if it isn't already available locally.

Docker's Compose examples similarly demonstrate defining one application service with build and another service using an existing image.

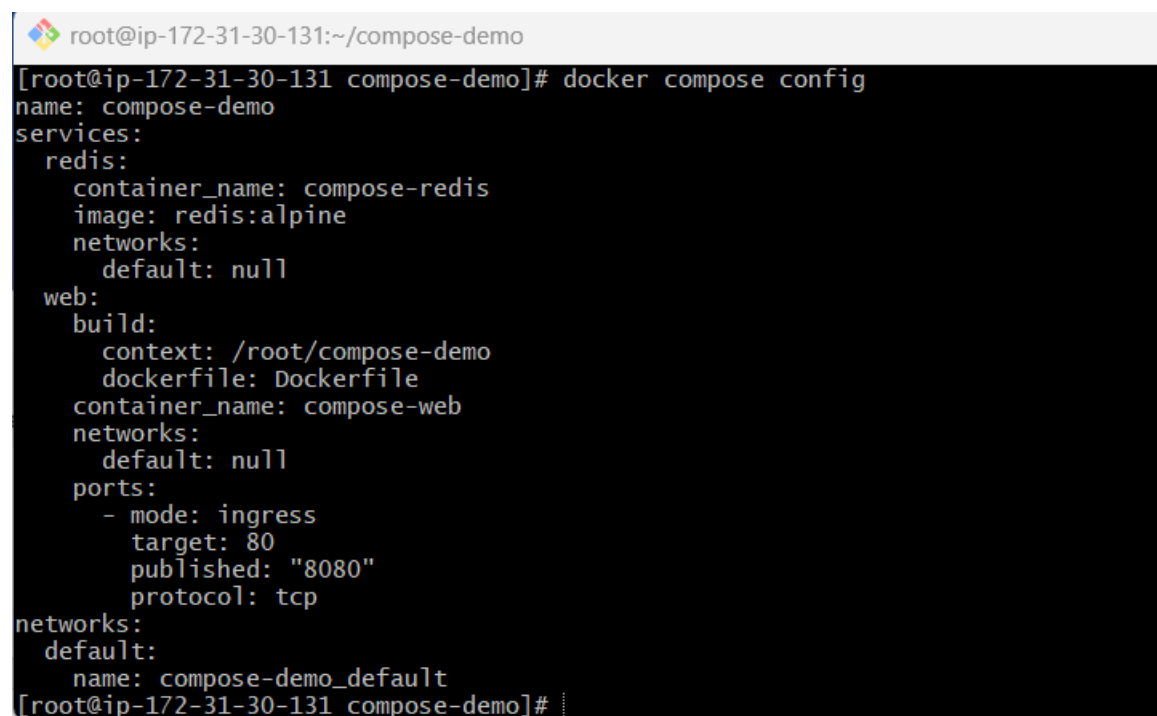
Part 4: Validate the Compose File

Before starting the application, validate the configuration:

```
docker compose config
```

If the configuration is valid, Docker Compose displays the resolved configuration.

This is useful for detecting YAML/configuration problems before starting the application.



```
root@ip-172-31-30-131:~/compose-demo
[root@ip-172-31-30-131 compose-demo]# docker compose config
name: compose-demo
services:
  redis:
    container_name: compose-redis
    image: redis:alpine
    networks:
      default: null
  web:
    build:
      context: /root/compose-demo
      dockerfile: Dockerfile
    container_name: compose-web
    networks:
      default: null
    ports:
      - mode: ingress
        target: 80
        published: "8080"
        protocol: tcp
networks:
  default:
    name: compose-demo_default
[root@ip-172-31-30-131 compose-demo]#
```

Part 5: Build the Application

10. Build the Images

Run:

```
docker compose build
```

Docker Compose will:

```
compose.yaml
```

|

↓

docker images

You should see the Nginx-based image and Redis image.

```
root@ip-172-31-30-131:~/compose-demo
[root@ip-172-31-30-131 compose-demo]# docker images
REPOSITORY          TAG       IMAGE ID       CREATED        SIZE
compose-demo-web    latest    6465537f4622   About a minute ago  162MB
multi-stage-multi    v1       f1941c3ca17c   43 minutes ago  282MB
multi-stage-single   v1       b106e3f4c768   About an hour ago  469MB
alpine-tomcat        1.0      96e1b50cb876   About an hour ago  227MB
my-nginx-running     v2       384c40c04463   4 hours ago     162MB
my-nginx-image       v1       39c541b70324   6 hours ago     162MB
[root@ip-172-31-30-131 compose-demo]#
```

Part 6: Start the Application

12. Start the Services

Run:

```
docker compose up -d
```

The -d option runs the containers in detached/background mode.

Docker's documentation uses `docker compose up -d` to start Compose applications in the background.

13. Check Running Services

Run:

```
docker compose ps
```

```
root@ip-172-31-30-131:~/compose-demo
[root@ip-172-31-30-131 compose-demo]# docker compose up -d
[+] up 3/3
✓ Network compose-demo_default Created 0.1s
✓ Container compose-web Started 0.7s
✓ Container compose-redis Started 0.6s
[root@ip-172-31-30-131 compose-demo]# docker compose ps
NAME                IMAGE              COMMAND                  SERVICE    CREATED        STATUS        PORTS
compose-redis       redis:alpine       "docker-entrypoint.s...  redis      14 seconds ago Up 12 seconds 6379/tcp
compose-web         compose-demo-web   "/docker-entrypoint....  web        14 seconds ago Up 12 seconds 0.0.0.0:8083->80/tcp, [::]:8083->80/tcp
[root@ip-172-31-30-131 compose-demo]#
```

14. Check the Application Logs

To see logs from all services:

```
docker compose logs
```

To continuously follow the logs:

```
docker compose logs -f
```

Docker Compose provides logs, ps, up, and down as core commands for managing the application lifecycle.

Press:

Ctrl + C

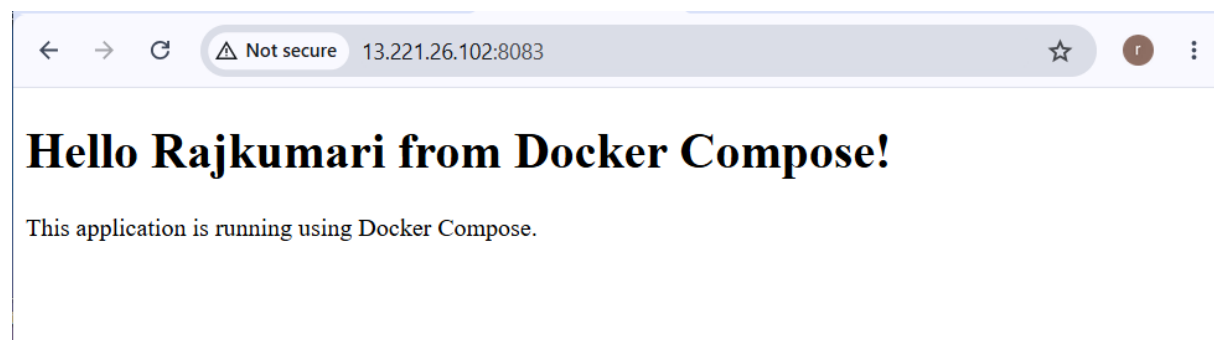
to stop following the logs.

Part 7: Test the Application

15. Test from the Docker Server

Run:

```
curl http://localhost:8080
```



Part 8: Verify Redis

17. Check Redis Container

Run:

```
docker compose ps
```

Confirm that:

```
compose-redis
```

is running.

You can also execute a Redis command inside the container:

```
docker exec -it compose-redis redis-cli ping
```


Expected:

PONG

This confirms that Redis is running successfully.

18. Check the Compose Network

Run:

```
docker network ls
```

You should see a network created for the Compose project.

Compose automatically creates a network for the application stack unless you configure a different networking setup.

Part 9: Useful Docker Compose Commands

Start Application

```
docker compose up -d
```

Stop Application

```
docker compose stop
```

Start Again

```
docker compose start
```

View Containers

```
docker compose ps
```

View Logs

```
docker compose logs
```

Follow Logs

```
docker compose logs -f
```

Build Images

```
docker compose build
```

Rebuild and Start

`docker compose up -d --build`

Execute Command Inside a Service

`docker compose exec web bash`

Stop and Remove Containers

`docker compose down`

`docker compose down` stops and removes the application's containers and other Compose-managed resources according to the configuration.

Part 10: Rebuild the Application

If you modify `index.html`, rebuild and recreate the application:

`docker compose up -d --build`

This is useful when you make changes to the Dockerfile or application files.

Part 11: Stop and Remove the Sample Application

When the assignment is complete:

`docker compose down`

Verify:

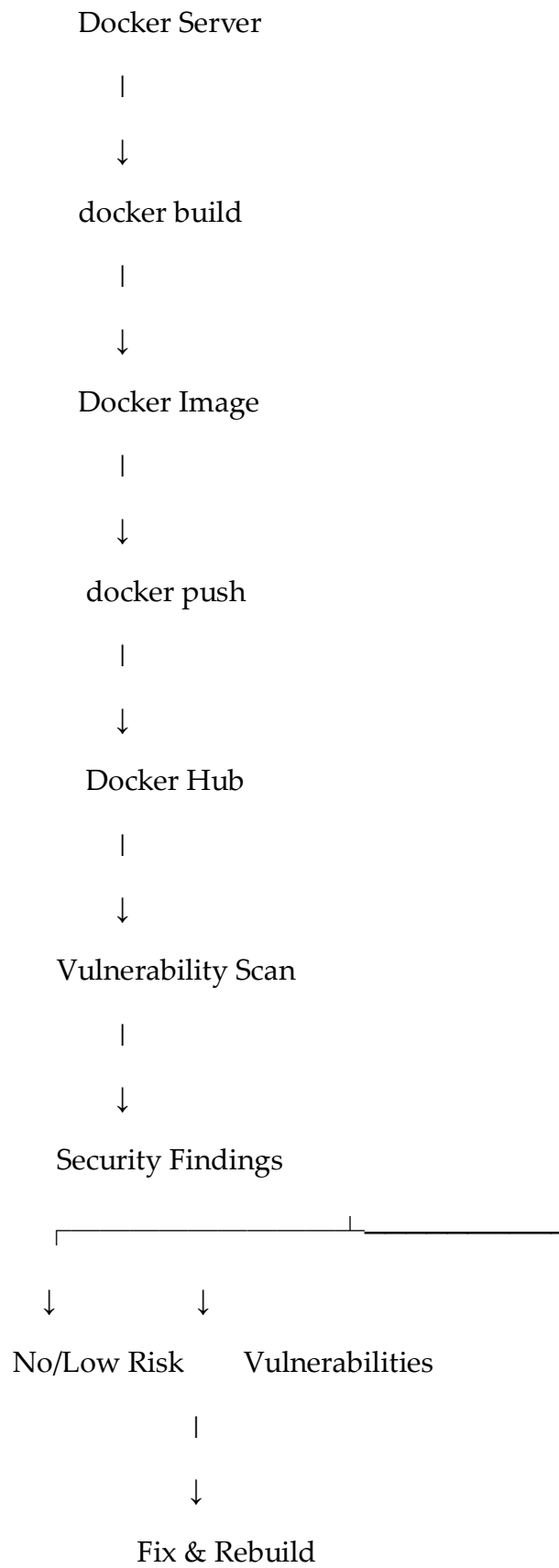
`docker compose ps`

The services should no longer be running.

6. Implement solution to scan images when pushed to docker hub registry.

1. Objective

The objective is to configure a Docker Hub repository so that Docker images are scanned for vulnerabilities after they are pushed.



2. Prerequisites

You need:

- Docker installed
- Docker Hub account
- Docker Hub repository
- Docker server/EC2 instance
- Docker Hub username
- Permission to push to the repository

Check Docker:

`docker --version`

Example:

Docker version 28.x.x

Check Docker login:

`docker login`

Enter:

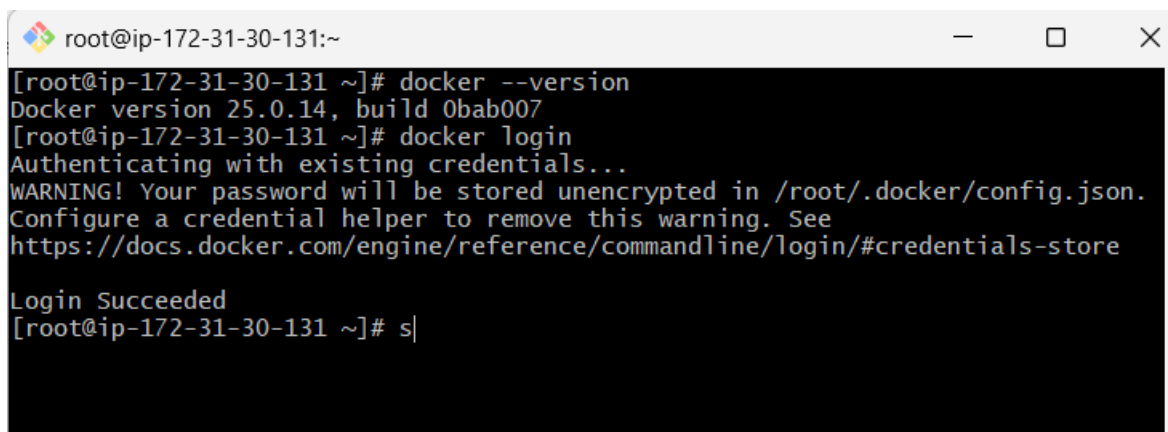
Username: <your-dockerhub-username>

Password: <your-password-or-access-token>

Expected:

Login Succeeded

For Docker Hub, using a **Personal Access Token (PAT)** instead of your Docker Hub password is the preferred authentication method.

A terminal window screenshot with a title bar showing 'root@ip-172-31-30-131:~'. The terminal output shows the command 'docker --version' returning 'Docker version 25.0.14, build 0bab007', followed by 'docker login' which authenticates with existing credentials, displays a warning about unencrypted passwords, and finally shows 'Login Succeeded'. The prompt returns to the shell.

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# docker --version  
Docker version 25.0.14, build 0bab007  
[root@ip-172-31-30-131 ~]# docker login  
Authenticating with existing credentials...  
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/engine/reference/commandline/login/#credentials-store  
  
Login Succeeded  
[root@ip-172-31-30-131 ~]# s|
```

3. Create a Docker Hub Repository

Go to Docker Hub:

[Docker Hub](https://hub.docker.com/)

After logging in:

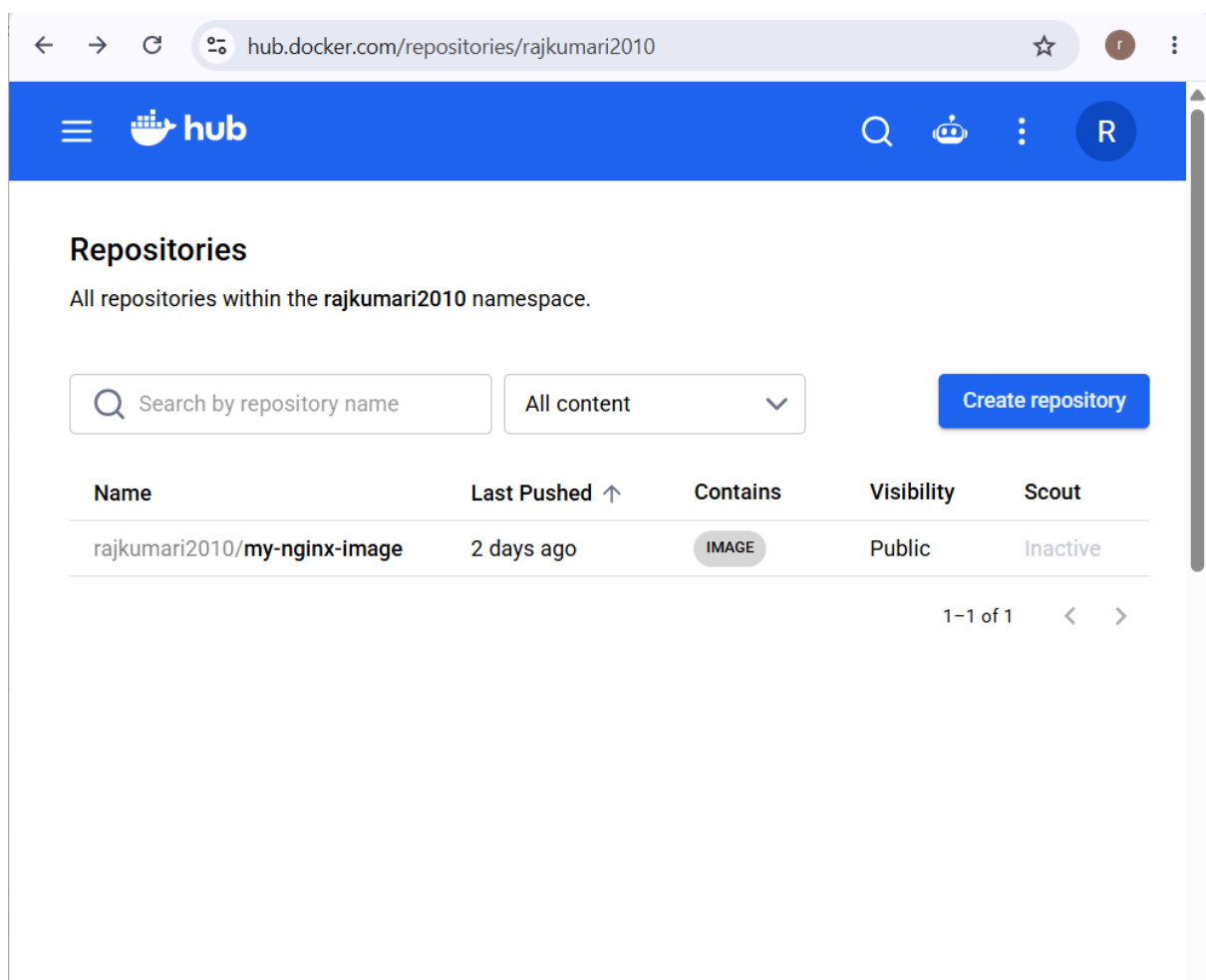
Docker Hub



Repositories



Create repository



Enter a repository name, for example:

docker-image-scan-demo

Choose the visibility:

Public

or:

Private

Then click:

Create

Your repository will look like:

<dockerhub-username>/docker-image-scan-demo

For example:

rajkumari/docker-image-scan-demo

The screenshot shows the Docker Hub 'Create repository' page. At the top is a blue header with the Docker Hub logo and navigation icons. Below the header, the page title is 'Repositories / Create'. A message states 'Using 0 of 1 private repositories. Upgrade your plan to add more'. The main section is titled 'Create repository'. It contains a text input field for 'Repository Name *' with the value 'docker-image-scan-demo'. Below this is a 'Short description' section with a text input field and a small icon of a Docker container. A note explains that the description is used for indexing and search. The 'Visibility' section shows two options: 'Public' (selected) and 'Private'. The 'Public' option is described as 'Appears in Docker Hub search results', and the 'Private' option is described as 'Only visible to you'. At the bottom right are 'Cancel' and 'Create' buttons. On the right side of the page, there is a 'Pushing images' section with instructions on how to push a new image to the repository using the CLI, including a code block with the commands: `docker tag local-image:tagname new-repo:tagname` and `docker push new-repo:tagname`. A note below the code block says 'Make sure to replace tagname with your desired image repository tag.'

Repositories / Create

Using 0 of 1 private repositories. [Upgrade your plan to add more](#)

Create repository

Repository Name *

 Short description

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. [Upgrade your plan to add more](#)

☒ Public  Appears in Docker Hub search results

☐ Private  Only visible to you

[Cancel](#) [Create](#)

Pushing images

You can push a new image to this repository using the CLI:

```
docker tag local-image:tagname new-repo:tagname
docker push new-repo:tagname
```

Make sure to replace `tagname` with your desired image repository tag.

The screenshot shows a web browser at the URL `hub.docker.com/repository/docker/rajkumari2010/docker-image-scan-demo/ge...`. The Docker Hub header is blue with the 'hub' logo and search, robot, and user icons. The breadcrumb trail is `Repositories / docker-image-scan-demo / General`. Below this, it says 'Using 0 of 1 private repositories.' The repository name **rajkumari2010/docker-image-scan-demo** is displayed with a warning icon. It was 'Created less than a minute ago' and has 0 stars and 0 downloads. There are links to 'Add a description' and 'Add a category', both with edit and info icons. Under the heading 'Docker commands', it instructs to 'push a new tag to this repository:' and shows the command `docker push rajkumari2010/docker-image-scan-demo:tagname` in a light gray box. A 'Public view' button is on the right. A tab bar at the bottom includes 'General' (selected), 'Tags', 'Image Management', 'Collaborators', 'Webhooks', and 'Settings'. The 'Tags' section shows 'Tags' with an 'INCOMPLETE' status and an 'Activate' link for 'DOCKER SCOUT INACTIVE'. A note at the bottom says 'Pushed images appear here.'

4. Create a Sample Docker Image

On your Docker server:

```
mkdir dockerhub-scan-demo
```

```
cd dockerhub-scan-demo
```

Create a Dockerfile:

```
vi Dockerfile
```

```
root@ip-172-31-30-131:~/dockerhub-scan-demo
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

5. Create Sample HTML File

Create:

vi index.html

```
root@ip-172-31-30-131:~/dockerhub-scan-demo
<!DOCTYPE html>
<html>
<head>
  <title>Docker Hub Image Scan</title>
</head>
<body>
  <h1>Rajkumari....|Docker Hub Vulnerability Scanning Demo</h1>
  <p>This Docker image is pushed to Docker Hub and scanned for vulnerabilities
.</p>
</body>
</html>
```

6. Build the Docker Image

Run:

docker build -t dockerhub-scan-demo:v1 .

Verify:

docker images

You should see:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
------------	-----	----------	---------	------

dockerhub-scan-demo v1 xxxxxxxxxxxx Few seconds ago ...

```
root@ip-172-31-30-131:~/dockerhub-scan-demo
[root@ip-172-31-30-131 dockerhub-scan-demo]# ls
Dockerfile
[root@ip-172-31-30-131 dockerhub-scan-demo]# vi Dockerfile
[root@ip-172-31-30-131 dockerhub-scan-demo]# vi index.html
[root@ip-172-31-30-131 dockerhub-scan-demo]# docker build -t dockerhub-scan-demo:v1 .
[+] Building 0.5s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 212B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:latest  0.2s
=> [auth] library/nginx:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                   0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 353B                                 0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:latest@sha256:b34848eff6db7 0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html      0.1s
=> exporting to image                                           0.0s
=> => exporting layers                                          0.0s
=> => writing image sha256:34b3ffb37895a030d272baba2511028710b7bc099378c 0.0s
=> => naming to docker.io/library/dockerhub-scan-demo:v1      0.0s
[root@ip-172-31-30-131 dockerhub-scan-demo]# |
```

```
root@ip-172-31-30-131:~/dockerhub-scan-demo
[root@ip-172-31-30-131 dockerhub-scan-demo]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
dockerhub-scan-demo v1          34b3ffb37895 About a minute ago 162MB
compose-demo-web    latest      6465537f4622 11 hours ago  162MB
multi-stage-multi    v1          f1941c3ca17c 12 hours ago  282MB
multi-stage-single   v1          b106e3f4c768 12 hours ago  469MB
alpine-tomcat        1.0         96e1b50cb876 12 hours ago  227MB
my-nginx-running     v2          384c40c04463 15 hours ago  162MB
my-nginx-image       v1          39c541b70324 17 hours ago  162MB
redis                alpine      00c30ddf0ef8 8 days ago    119MB
[root@ip-172-31-30-131 dockerhub-scan-demo]# |
```

7. Test the Image Locally

Before pushing the image, test it.

Run:

```
docker run -d \
```

```
--name dockerhub-scan-container \
```

```
-p 8080:80 \
```

```
dockerhub-scan-demo:v1
```



Rajkumari.....Docker Hub Vulnerability Scanning Demo

This Docker image is pushed to Docker Hub and scanned for vulnerabilities.

`docker stop dockerhub-scan-container`

`docker rm dockerhub-scan-container`

```
root@ip-172-31-30-131:~/dockerhub-scan-demo
[root@ip-172-31-30-131 dockerhub-scan-demo]# docker run -d --name dockerhub-scan-container -p 8082:80 docker-scan-demo:v1
92aa8473bbd9baa392f13f2ba5d145c44c34b50c896e21cad1a2834dbd2c1fe6
[root@ip-172-31-30-131 dockerhub-scan-demo]# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
92aa8473bbd9   docker-scan-demo:v1                "/docker-entrypoint..." 8 seconds ago  Up
7 seconds     0.0.0.0:8082->80/tcp, :::8082->80/tcp  dockerhub-scan-container
[root@ip-172-31-30-131 dockerhub-scan-demo]# docker stop dockerhub-scan-container
dockerhub-scan-container
[root@ip-172-31-30-131 dockerhub-scan-demo]# docker rm dockerhub-scan-container
dockerhub-scan-container
[root@ip-172-31-30-131 dockerhub-scan-demo]# |
```

8. Login to Docker Hub

Run:

`docker login`

You will be prompted:

Username:

Password:

Enter your Docker Hub username and Personal Access Token.

Expected:

Login Succeeded

You can verify Docker's stored login configuration:

```
cat ~/.docker/config.json
```

Be careful not to share this file because it contains Docker authentication information.

```
root@ip-172-31-30-131:~/dockerhub-scan-demo
[root@ip-172-31-30-131 dockerhub-scan-demo]# docker tag dockerhub-scan-demo:v1 r
ajkumari2010/docker-image-scan-demo:v1
[root@ip-172-31-30-131 dockerhub-scan-demo]# docker images
REPOSITORY                                TAG      IMAGE ID      CREATED
SIZE
docker-scan-demo                          v1       34b3ffb37895  19 minutes ago
162MB
dockerhub-scan-demo                      v1       34b3ffb37895  19 minutes ago
162MB
rajkumari2010/docker-image-scan-demo     v1       34b3ffb37895  19 minutes ago
162MB
compose-demo-web                         latest   6465537f4622  11 hours ago
162MB
multi-stage-multi                        v1       f1941c3ca17c  12 hours ago
282MB
multi-stage-single                       v1       b106e3f4c768  12 hours ago
469MB
alpine-tomcat                           1.0      96e1b50cb876  13 hours ago
227MB
my-nginx-running                         v2       384c40c04463  15 hours ago
162MB
my-nginx-image                           v1       39c541b70324  17 hours ago
162MB
redis                                    alpine   00c30ddf0ef8  8 days ago
119MB
[root@ip-172-31-30-131 dockerhub-scan-demo]# |
```

10. Push the Image to Docker Hub

Run: `docker push rajkumari2010/docker-image-scan-demo:v1`

```
root@ip-172-31-30-131:~/dockerhub-scan-demo
[root@ip-172-31-30-131 dockerhub-scan-demo]# docker push rajkumari2010/docker-im
age-scan-demo:v1
The push refers to repository [docker.io/rajkumari2010/docker-image-scan-demo]
3b9d93e77d9f: Pushed
a8fae4102358: Mounted from library/nginx
b0020123c41b: Mounted from library/nginx
f29840e7d6e5: Mounted from library/nginx
f109061e6486: Mounted from library/nginx
e8fb5de3ef79: Mounted from library/nginx
070ef859f070: Mounted from library/nginx
411a86676185: Mounted from library/nginx
v1: digest: sha256:e89b701d21fa25c920baedddae06ce6118c00d7541421dd9d76663e74e2cc
0ef size: 1985
[root@ip-172-31-30-131 dockerhub-scan-demo]# |
```

11. Verify the Image in Docker Hub

Open:

Docker Hub Repositories

Navigate to:

Repositories



docker-image-scan-demo

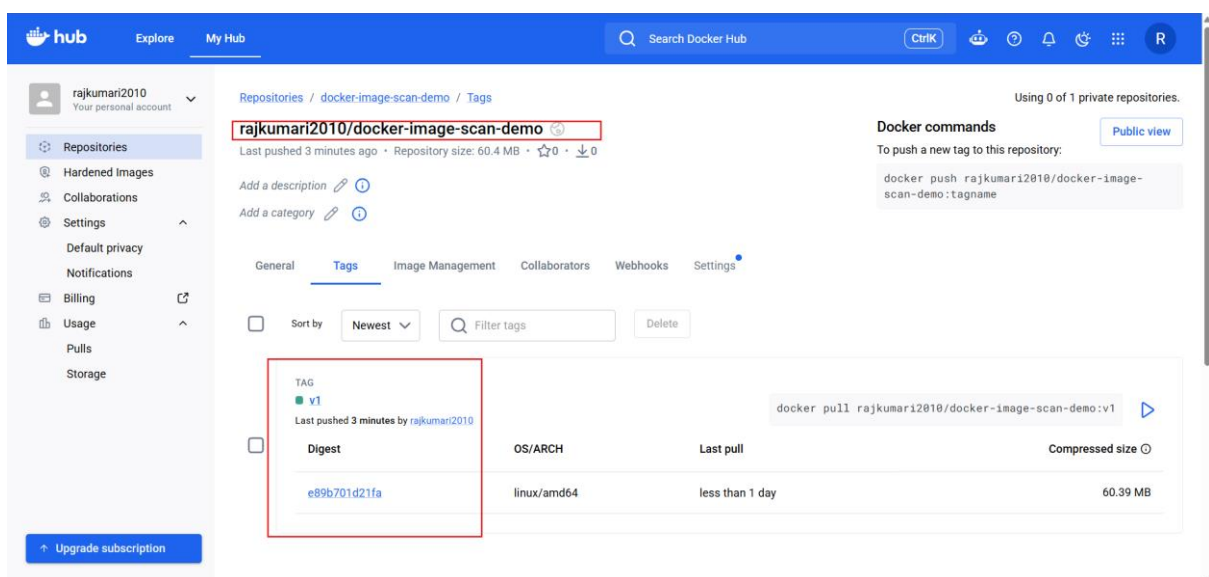
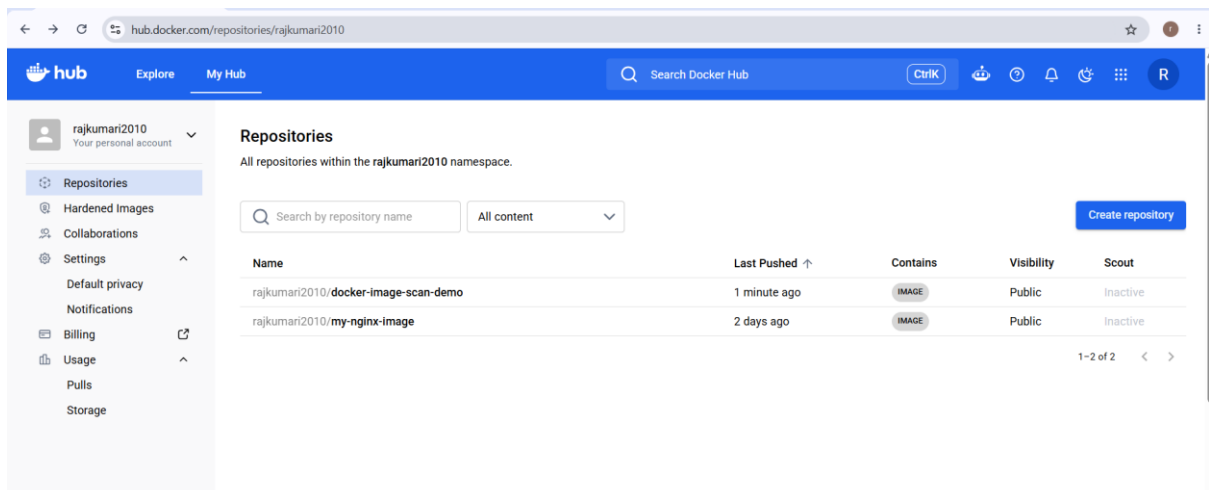


Tags



v1

You should see the pushed image.



12. Enable/Check Vulnerability Scanning

Open the repository in Docker Hub.

Look for a section such as:

Vulnerabilities

or:

Scan

Depending on your Docker Hub plan/UI, vulnerability scanning may appear at the repository or image/tag level.

The expected workflow is:

Docker Hub Repository

|

↓

Image Tag

|

↓

Vulnerabilities / Scan

|

↓

Security Findings

If scanning is available for your repository, Docker Hub will analyze the image after it has been pushed.

The screenshot shows the Docker Hub interface for a repository named 'rajkumari2010/docker-image-scan-demo:v1'. The page is divided into several sections:

- Header:** Docker Hub logo, navigation links (Explore, My Hub), search bar, and user profile (rajkumari2010).
- Repository Info:** Repository name, manifest digest, OS/ARCH (linux/amd64), compressed size (60.39 MB), last pushed (1 minute by rajkumari2010), type (Image), and a vulnerability score (4.9/10).
- Image Layers:** A table showing the layers of the image. The first layer is 'nginx:1'. The second layer is 'COPY index.html /usr/share/nginx/html/index.html'. The third layer is 'EXPOSE map[80/tcp:[]]'. The fourth layer is 'CMD ["nginx" "-g" "daemon off;"]'.
- Vulnerabilities:** A table showing 91 vulnerabilities. The table has columns for CVE ID, Severity, Fixable, Present in, and Affect... The vulnerabilities are listed in descending order of severity score.

7. Implement solution to scan images when pushed to AWS ECR.

The complete flow is:

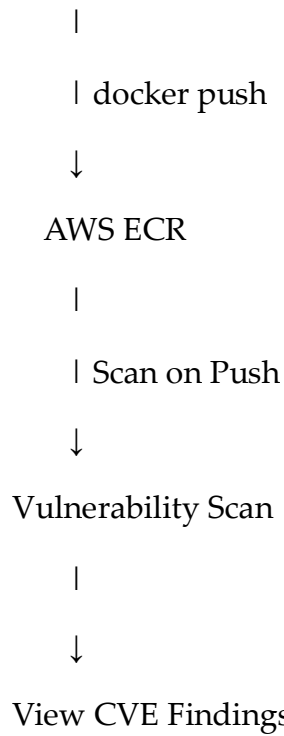
Docker Server / EC2

|

| docker build

↓

Docker Image



1. Prerequisites

Make sure Docker and AWS CLI are installed.

Check Docker

```
docker --version
```

Check AWS CLI

```
aws --version
```

Verify AWS credentials

```
aws sts get-caller-identity
```

You should see your AWS Account ID and IAM role/user

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# aws --version  
aws-cli/2.33.15 Python/3.9.25 Linux/6.18.39-79.141.amzn2023.x86_64 source/x86_64  
.amzn.2023  
[root@ip-172-31-30-131 ~]# aws sts get-caller-identity  
{  
  "UserId": "AROAYKI7M6OPWVOJDCRB:i-092add6ce1e26dacf",  
  "Account": "571833381790",  
  "Arn": "arn:aws:sts::571833381790:assumed-role/EC2-ECR-Role/i-092add6ce1e26d  
acf"  
}  
[root@ip-172-31-30-131 ~]#  
[root@ip-172-31-30-131 ~]# |
```

2. Create an ECR Repository

We will create a repository called:

docker-scan-demo

Run:

```
aws ecr create-repository \  
  --repository-name docker-scan-demo \  
  --region us-east-1
```

Verify:

```
aws ecr describe-repositories \  
  --repository-names docker-scan-demo \  
  --region us-east-1
```

Your repository URL will look like:

123456789012.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo

Replace 123456789012 with your AWS Account ID.


```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# aws ecr create-repository \  
  --repository-name docker-scan-demo \  
  --region us-east-1  
{  
  "repository": {  
    "repositoryArn": "arn:aws:ecr:us-east-1:571833381790:repository/docker-scan-demo",  
    "registryId": "571833381790",  
    "repositoryName": "docker-scan-demo",  
    "repositoryUri": "571833381790.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo",  
    "createdAt": "2026-08-27T06:19:26.310000+00:00",  
    "imageTagMutability": "MUTABLE",  
    "imageScanningConfiguration": {  
      "scanOnPush": false  
    },  
    "encryptionConfiguration": {  
      "encryptionType": "AES256"  
    }  
  }  
}  
[root@ip-172-31-30-131 ~]# |
```

Verify:

```
aws ecr describe-repositories \  
  --repository-names docker-scan-demo \  
  --region us-east-1
```

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# aws ecr describe-repositories \  
  --repository-names docker-scan-demo \  
  --region us-east-1  
{  
  "repositories": [  
    {  
      "repositoryArn": "arn:aws:ecr:us-east-1:571833381790:repository/docker-scan-demo",  
      "registryId": "571833381790",  
      "repositoryName": "docker-scan-demo",  
      "repositoryUri": "571833381790.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo",  
      "createdAt": "2026-08-27T06:19:26.310000+00:00",  
      "imageTagMutability": "MUTABLE",  
      "imageScanningConfiguration": {  
        "scanOnPush": false  
      },  
      "encryptionConfiguration": {  
        "encryptionType": "AES256"  
      }  
    }  
  ]  
}  
[root@ip-172-31-30-131 ~]# |
```

3. Enable ECR Scan on Push

You can enable scanning from the AWS Console.

Go to:

AWS Console



Amazon ECR



Private registry



Scanning

Select:

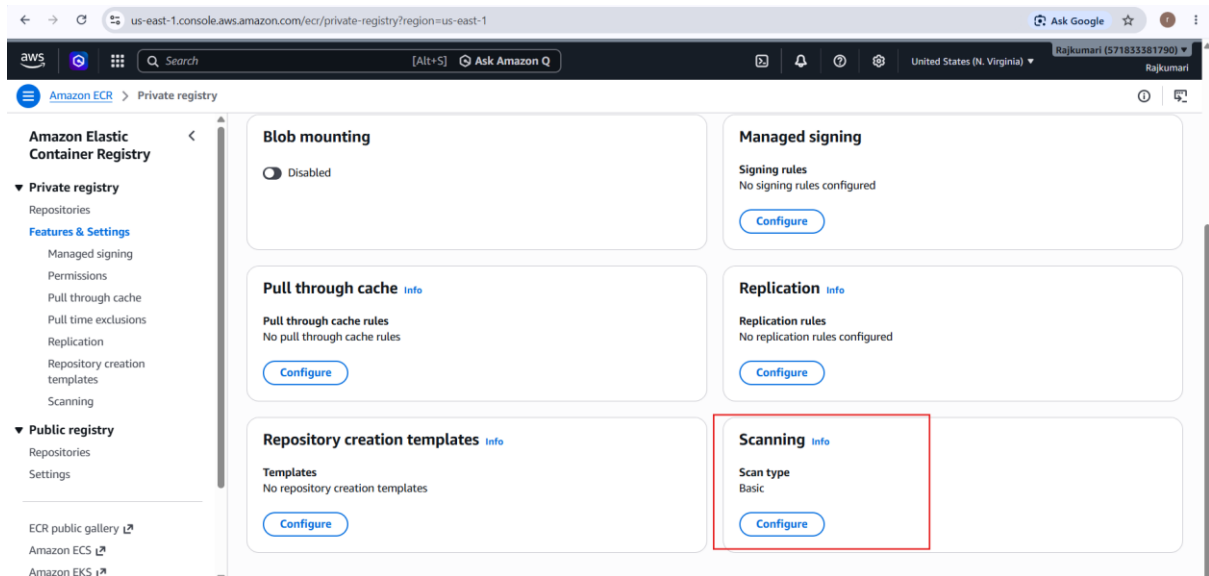
Basic scanning

Configure:

Scan on push

You can apply it to all repositories or configure a repository filter.

AWS ECR Basic Scanning supports automatic scanning when images are pushed to the registry.



The screenshot displays the Amazon Elastic Container Registry (ECR) console. The top navigation bar shows the AWS logo, a search icon, and the user profile 'Rajkumari (571833381790)'. The left sidebar contains the navigation menu with 'Amazon Elastic Container Registry' and 'Private registry' expanded. The main content area shows the 'Scanning' settings for a Private registry. The 'Scan type' section has two options: 'Basic scanning' (selected) and 'Enhanced scanning'. The 'Scan on push filters' section has a checkbox 'Scan on push all repositories' which is checked. There is an 'Add filter' button and a list of filters (currently empty). At the bottom right, there are 'Cancel' and 'Save' buttons.

container images.

Scan type
Select the scanning type that will be used for this registry. Enhanced scanning is a paid service. [Learn more](#).

☒ **Basic scanning**
Basic scanning allows manual scans and scan on push of images in this registry. This is a free service.

☐ **Enhanced scanning**
Enhanced scanning with Amazon Inspector provides automated continuous scanning. Inspector identifies vulnerabilities in both operating system and programming language (such as Python, Java, Ruby etc.) packages in real time.

Scan on push filters
Select which repositories to scan for vulnerabilities on image push. Filters with no wildcard will match all repository names that contain the filter. Filters with wildcards (*) will match on a repository name where the wildcard replaces zero or more characters in the repository name.

☒ Scan on push all repositories

4. Enable Scan on Push Using AWS CLI

You can also configure it from the command line.

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# aws ecr put-registry-scanning-configuration \  
--scan-type BASIC \  
--rules '[  
  {  
    "scanFrequency": "SCAN_ON_PUSH",  
    "repositoryFilters": [  
      {  
        "filter": "docker-scan-demo",  
        "filterType": "WILDCARD"  
      }  
    ]  
  }  
' \  
--region us-east-1  
{  
  "registryScanningConfiguration": {  
    "scanType": "BASIC",  
    "rules": [  
      {  
        "scanFrequency": "SCAN_ON_PUSH",  
        "repositoryFilters": [  
          {  
            "filter": "docker-scan-demo",  
            "filterType": "WILDCARD"  
          }  
        ]  
      }  
    ]  
  }  
}
```

Verify:

```
aws ecr get-registry-scanning-configuration \  
--region us-east-1
```

You should see:

scanType: BASIC

scanFrequency: SCAN_ON_PUSH

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# aws ecr get-registry-scanning-configuration \  
--region us-east-1  
{  
  "registryId": "571833381790",  
  "scanningConfiguration": {  
    "scanType": "BASIC",  
    "rules": [  
      {  
        "scanFrequency": "SCAN_ON_PUSH",  
        "repositoryFilters": [  
          {  
            "filter": "docker-scan-demo",  
            "filterType": "WILDCARD"  
          }  
        ]  
      }  
    ]  
  }  
}  
[root@ip-172-31-30-131 ~]# |
```

5. Create a Sample Docker Image

Create a directory:

```
mkdir docker-scan-demo
```

```
cd docker-scan-demo
```

Create the Dockerfile:

```
vi Dockerfile
```

Add:

```
FROM nginx:latest
```

```
COPY index.html /usr/share/nginx/html/index.html
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

```
root@ip-172-31-30-131:~/docker-scan-demo
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

6. Create index.html

vi index.html

```
root@ip-172-31-30-131:~/docker-scan-demo
<!DOCTYPE html>
<html>
<head>
  <title>ECR Image Scanning Demo</title>
</head>
<body>
  <h1>Rajkumari...AWS ECR Image Scanning Demo</h1>
  <p>This image will be automatically scanned after being pushed to ECR.</p>
</body>
</html>
```

"index.html" 10L, 237B

7. Build the Docker Image

Run:

docker build -t docker-scan-demo:v1 .

Verify:

docker images

You should see:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
docker-scan-demo	v1	xxxxxxxxxxxx	Few seconds ago ...	

```
root@ip-172-31-30-131:~/docker-scan-demo
[root@ip-172-31-30-131 ~]# mkdir docker-scan-demo
cd docker-scan-demo
[root@ip-172-31-30-131 docker-scan-demo]# vi Dockerfile
[root@ip-172-31-30-131 docker-scan-demo]# vi index.html
[root@ip-172-31-30-131 docker-scan-demo]# vi index.html
[root@ip-172-31-30-131 docker-scan-demo]# docker build -t docker-scan-demo:v2 .
[+] Building 0.4s (8/8) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 212B 0.0s
=> [internal] load metadata for docker.io/library/nginx:latest 0.2s
=> [auth] library/nginx:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 336B 0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:latest@sha256:b34848eff6db7 0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html 0.0s
=> exporting to image 0.0s
=> => exporting layers 0.0s
=> => writing image sha256:071c993a79a78ecb83f3d09938830af0d676b6cce349e 0.0s
=> => naming to docker.io/library/docker-scan-demo:v2 0.0s
[root@ip-172-31-30-131 docker-scan-demo]# docker images
REPOSITORY              TAG          IMAGE ID          CREATED
SIZE
docker-scan-demo        v2           071c993a79a7     24 seconds ago
162MB
docker-scan-demo        v1           34b3ffb37895     About an hour ag
o 162MB
dockerhub-scan-demo     v1           34b3ffb37895     About an hour ag
o 162MB
rajkumari2010/docker-image-scan-demo v1           34b3ffb37895     About an hour ag
o 162MB
compose-demo-web        latest       6465537f4622     12 hours ago
162MB
multi-stage-multi       v1           f1941c3ca17c     13 hours ago
282MB
multi-stage-single      v1           b106e3f4c768     13 hours ago
469MB
alpine-tomcat           1.0         96e1b50cb876     14 hours ago
227MB
my-nginx-running        v2           384c40c04463     16 hours ago
162MB
my-nginx-image          v1           39c541b70324     19 hours ago
162MB
redis                   alpine       00c30ddf0ef8     8 days ago
119MB
[root@ip-172-31-30-131 docker-scan-demo]# |
```

8. Test the Image Locally

Run:

docker run -d \

--name docker-scan-container \

-p 8080:80 \

docker-scan-demo:v1

Check:

docker ps

```
root@ip-172-31-30-131:~/docker-scan-demo
[root@ip-172-31-30-131 docker-scan-demo]# docker run -d --name docker-scan-ecr-cont -p 8082:80 docker-scan-demo:v2
24eca497b195c92adaf790ac4271ef1ba7c58c93427c4013a552c40536bd59f7
[root@ip-172-31-30-131 docker-scan-demo]# docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS
24eca497b195   docker-scan-demo:v2                "/docker-entrypoint..."              5 seconds ago   Up
4 seconds     0.0.0.0:8082->80/tcp, :::8082->80/tcp  docker-scan-ecr-cont
[root@ip-172-31-30-131 docker-scan-demo]#
```



Rajkumari....AWS ECR Image Scanning Demo

This image will be automatically scanned after being pushed to ECR.

You should receive the HTML page.

After testing:

```
docker stop docker-scan-container
```

```
docker rm docker-scan-container
```

9. Authenticate Docker with ECR

Run: 9. Authenticate Docker with ECR

Run:

```
aws ecr get-login-password --region us-east-1 | \
```

```
docker login \
```



```
--username AWS \
```

```
--password-stdin \
```

```
123456789012.dkr.ecr.us-east-1.amazonaws.com
```

Replace with your account ID

10. Tag the Image

Tag the image with your ECR repository URL.

```
docker tag docker-scan-demo:v1 \
```

```
123456789012.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo:v1
```

Verify:

```
docker images
```

You should now see:

```
docker-scan-demo
```

```
    v1
```

```
123456789012.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo
```

```
    v1
```

11. Push the Image to ECR

Run:

```
docker push \
```

```
123456789012.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo:v1
```

```

root@ip-172-31-30-131:~/docker-scan-demo
[root@ip-172-31-30-131 docker-scan-demo]# docker tag docker-scan-demo:v1 \
571833381790.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo:v1
[root@ip-172-31-30-131 docker-scan-demo]# aws ecr get-login-password --region us
-east-1 | \
docker login --username AWS --password-stdin \
571833381790.dkr.ecr.us-east-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-30-131 docker-scan-demo]# docker push \
571833381790.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo:v1
The push refers to repository [571833381790.dkr.ecr.us-east-1.amazonaws.com/dock
er-scan-demo]
3b9d93e77d9f: Pushed
a8fae4102358: Pushed
b0020123c41b: Pushed
f29840e7d6e5: Pushed
f109061e6486: Pushed
e8fb5de3ef79: Pushed
070ef859f070: Pushed
411a86676185: Pushed
v1: digest: sha256:91c0f047e467082097075b9336c375de7d691b60c39a518a6bb4b71deeadb
86c size: 1985
[root@ip-172-31-30-131 docker-scan-demo]# docker images
REPOSITORY          TAG          IMAGE
ID                  CREATED      SIZE
docker-scan-demo    v2           071c99
3a79a7             11 minutes ago  162MB
571833381790.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo v1          34b3ff
b37895             2 hours ago    162MB
docker-scan-demo    v1           34b3ff
b37895             2 hours ago    162MB
dockerhub-scan-demo v1           34b3ff
b37895             2 hours ago    162MB
rajkumar12010/docker-image-scan-demo v1          34b3ff
b37895             2 hours ago    162MB
compose-demo-web    latest       646553
7f4622             12 hours ago   162MB
multi-stage-multi    v1           f1941c
3ca17c             13 hours ago   282MB
multi-stage-single    v1           b106e3
f4c768             13 hours ago   469MB
alpine-tomcat         1.0          96e1b5
0cb876             14 hours ago   227MB
my-nginx-running     v2           384c40
c04463             16 hours ago   162MB
my-nginx-image       v1           39c541
b70324             19 hours ago   162MB
redis                alpine       00c30d
df0ef8             8 days ago     119MB
[root@ip-172-31-30-131 docker-scan-demo]#

```

12. Check the Image in ECR

Go to:

AWS Console



Amazon ECR



Repositories



docker-scan-demo

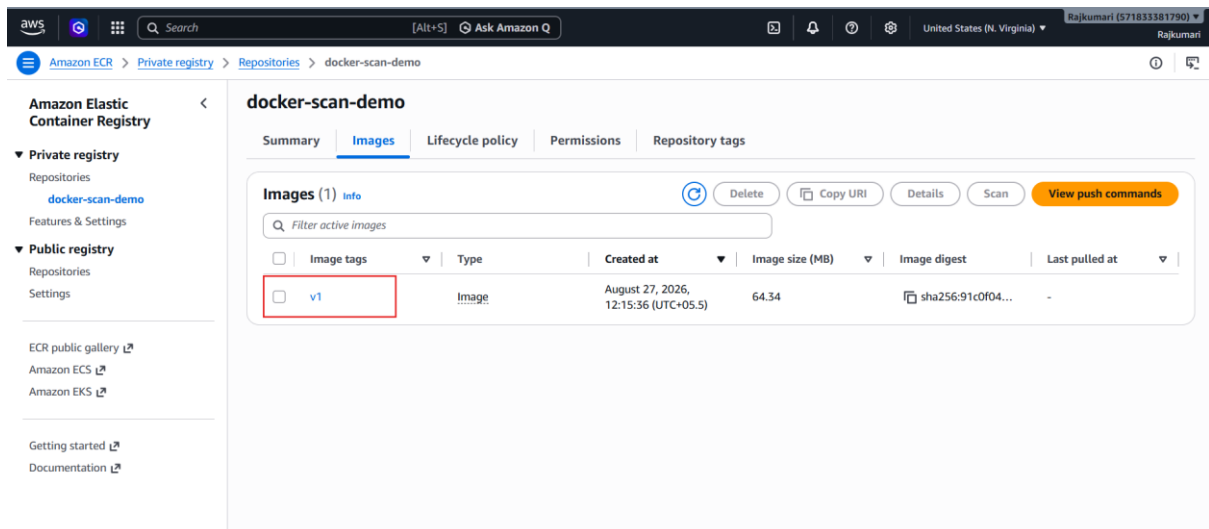
You should see:

Image tag: v1

Click/select the image.

Look for the vulnerability/security information.

The scan may initially be in progress.



13. View Vulnerability Findings

After the scan completes, ECR will show vulnerability findings.

You may see:

CRITICAL

HIGH

MEDIUM

LOW

For example:

CRITICAL 0

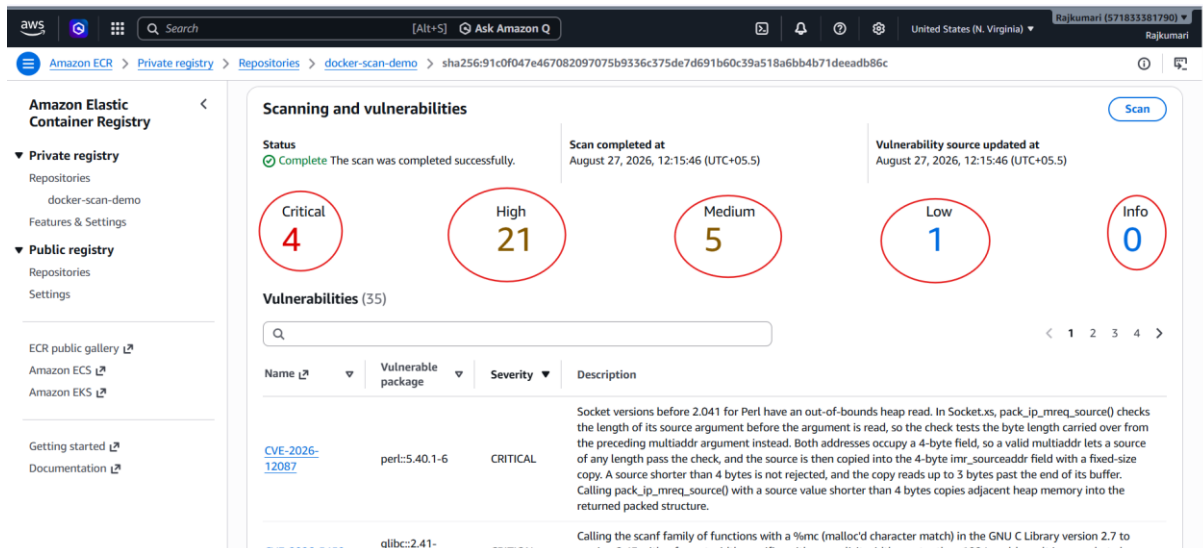
HIGH 2

MEDIUM 5

LOW 3

The actual numbers will depend on the image and current vulnerability database.

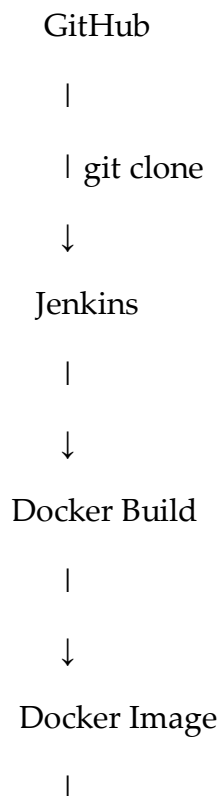
Basic ECR scanning primarily identifies vulnerabilities in the operating-system packages contained in the image.

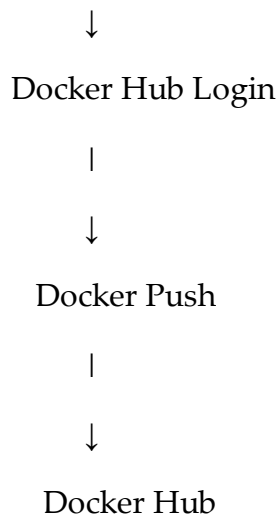


8. Create a Jenkins pipeline to create a docker image and push the image to Docker hub.

This is a very common **CI/CD practical task**. Since you are working with Jenkins and Docker, I'll show you a complete setup you can use for your documentation and practice.

Architecture





1. Prerequisites

Make sure you have:

- Jenkins installed
- Docker installed on the Jenkins server/agent
- Git installed
- A GitHub repository containing a Dockerfile
- Docker Hub account
- Docker Hub Personal Access Token
- Jenkins Docker Pipeline support/plugins

Check Docker on the Jenkins node:

```
docker --version
```

Check Git:

```
git --version
```

Check Jenkins user can execute Docker:

```
sudo -u jenkins docker ps
```

If you get:

permission denied

add Jenkins to the Docker group:

```
sudo usermod -aG docker jenkins
```

Then restart Jenkins:

```
sudo systemctl restart jenkins
```

You may need to reconnect/restart the Jenkins agent as well.

Verify again:

```
sudo -u jenkins docker ps
```

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# systemctl status jenkins  
● jenkins.service - Jenkins Continuous Integration Server  
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: >  
   Active: active (running) since Thu 2026-08-27 07:12:08 UTC; 18s ago  
   Main PID: 9140 (java)  
     Tasks: 42 (limit: 1059)  
    Memory: 330.5M  
       CPU: 20.402s  
   CGroup: /system.slice/jenkins.service  
           └─9140 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java>  
  
Aug 27 07:12:03 ip-172-31-30-131.ec2.internal jenkins[9140]: [LF]>  
Aug 27 07:12:03 ip-172-31-30-131.ec2.internal jenkins[9140]: [LF]> *****>  
Aug 27 07:12:03 ip-172-31-30-131.ec2.internal jenkins[9140]: [LF]> *****>  
Aug 27 07:12:03 ip-172-31-30-131.ec2.internal jenkins[9140]: [LF]> *****>  
Aug 27 07:12:08 ip-172-31-30-131.ec2.internal jenkins[9140]: 2026-08-27 07:12:0>  
Aug 27 07:12:08 ip-172-31-30-131.ec2.internal jenkins[9140]: 2026-08-27 07:12:0>  
Aug 27 07:12:08 ip-172-31-30-131.ec2.internal systemd[1]: Started jenkins.servi>  
Aug 27 07:12:08 ip-172-31-30-131.ec2.internal jenkins[9140]: 2026-08-27 07:12:0>  
Aug 27 07:12:08 ip-172-31-30-131.ec2.internal jenkins[9140]: 2026-08-27 07:12:0>  
Aug 27 07:12:13 ip-172-31-30-131.ec2.internal jenkins[9140]: 2026-08-27 07:12:1>  
lines 1-20/20 (END)
```

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# sudo -u jenkins docker ps  
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.44/containers/json": dial unix /var/run/docker.sock: connect: permission denied  
[root@ip-172-31-30-131 ~]# sudo usermod -aG docker jenkins  
[root@ip-172-31-30-131 ~]# sudo systemctl restart jenkins  
[root@ip-172-31-30-131 ~]# sudo -u jenkins docker ps  
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES  
[root@ip-172-31-30-131 ~]# |
```

2. Create Docker Hub Repository

Go to Docker Hub:

[Docker Hub](#)

Create a repository.

For example:

Repository name:

jenkins-docker-demo

Your image name will be:

<dockerhub-username>/jenkins-docker-demo

For example:

Rajkumari2010/jenkins-docker-demo

Repositories / Create

Using 0 of 1 private repositories. [Upgrade your plan to add more](#)

Create repository

Repository Name *

 Short description

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. [Upgrade your plan to add more](#)

☒ **Public**  Appears in Docker Hub search results

☐ **Private**  Only visible to you

[Cancel](#) [Create](#)

Pushing images

You can push a new image to this repository using the CLI:

```
docker tag local-image:tagname new-repo:tagname
docker push new-repo:tagname
```

Make sure to replace `tagname` with your desired image repository tag.

3. Create Docker Hub Access Token

Do **not** put your Docker Hub password directly inside the Jenkinsfile.

Use a Docker Hub Personal Access Token.

In Docker Hub, go to your account security settings and create an access token.

Example:

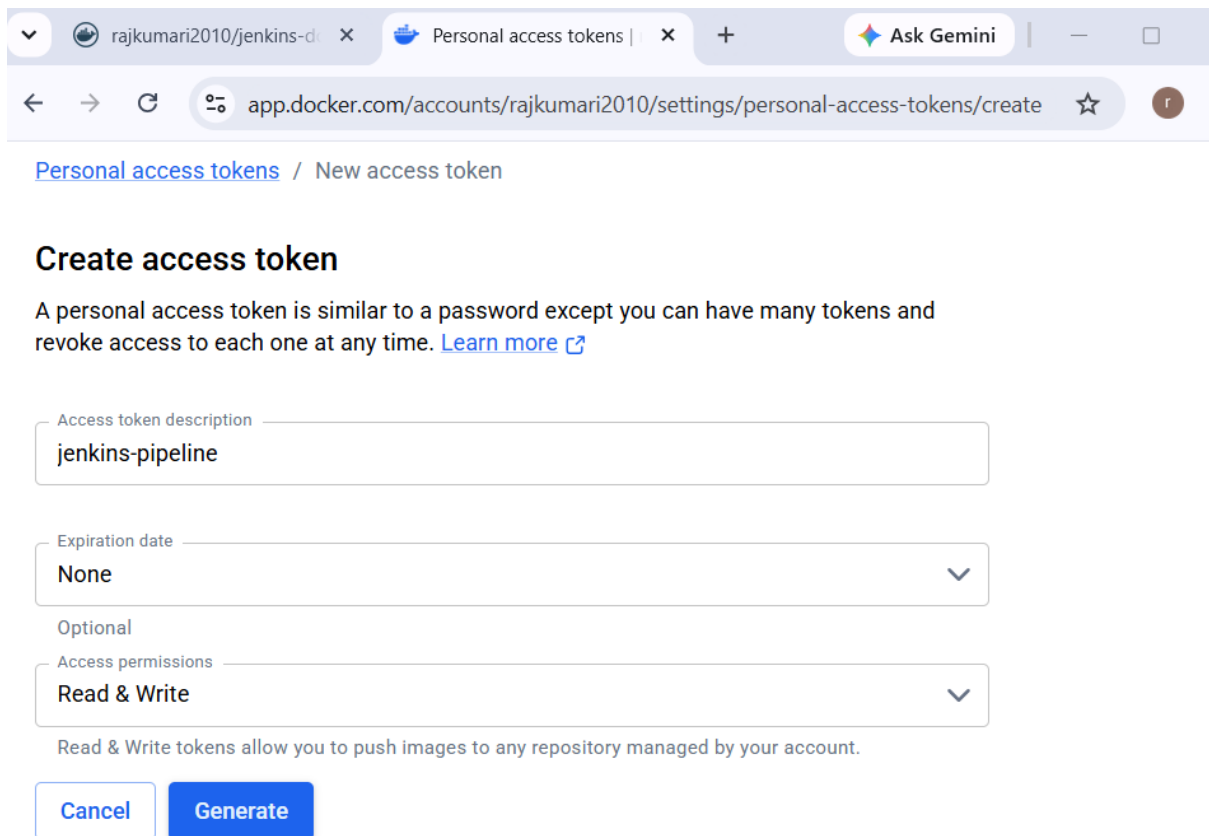
Username:

rajkumari

Token:

dckr_pat_xxxxxxxxxxxxxx

Keep the token secret.



The screenshot shows the 'Create access token' page on Docker Hub. The browser address bar shows the URL: `app.docker.com/accounts/rajkumari2010/settings/personal-access-tokens/create`. The page title is 'Personal access tokens / New access token'. Below the title, there is a section 'Create access token' with a description: 'A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)'. There are three input fields: 'Access token description' with the value 'jenkins-pipeline', 'Expiration date' with the value 'None', and 'Access permissions' with the value 'Read & Write'. Below these fields, there is a note: 'Read & Write tokens allow you to push images to any repository managed by your account.' At the bottom, there are two buttons: 'Cancel' and 'Generate'.

Personal access tokens / New access token

Create access token

A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)

Access token description
jenkins-pipeline

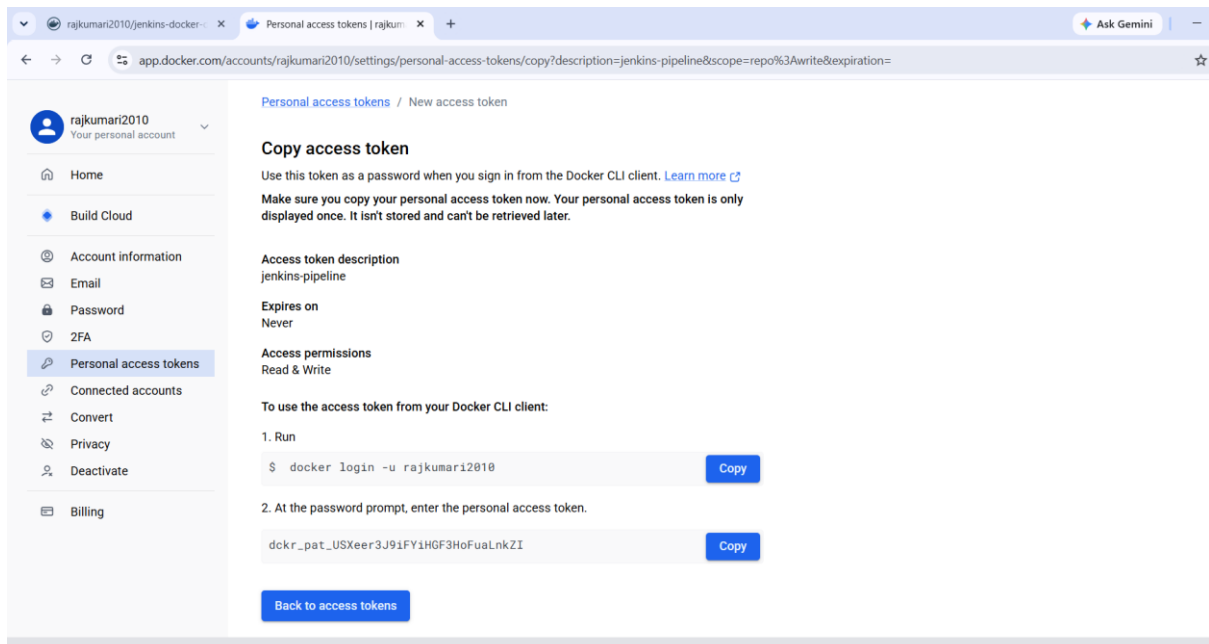
Expiration date
None

Optional

Access permissions
Read & Write

Read & Write tokens allow you to push images to any repository managed by your account.

[Cancel](#) [Generate](#)



The screenshot shows the 'Copy access token' page on Docker Hub. The browser address bar shows the URL: `app.docker.com/accounts/rajkumari2010/settings/personal-access-tokens/copy?description=jenkins-pipeline&scope=repo%3Awrite&expiration=`. The page title is 'Personal access tokens / New access token'. On the left, there is a sidebar with the user's account information and a list of settings: Home, Build Cloud, Account information, Email, Password, 2FA, Personal access tokens (selected), Connected accounts, Convert, Privacy, Deactivate, and Billing. The main content area has the title 'Copy access token' and a warning: 'Use this token as a password when you sign in from the Docker CLI client. [Learn more](#). Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.' Below this, there are three sections: 'Access token description' with the value 'jenkins-pipeline', 'Expires on' with the value 'Never', and 'Access permissions' with the value 'Read & Write'. There is a section 'To use the access token from your Docker CLI client:' with two steps: 1. Run `$ docker login -u rajkumari2010` (with a 'Copy' button) and 2. At the password prompt, enter the personal access token. The token is displayed as `dckr_pat_USXeer3J91FY1HGF3HoFuaLnkZI` (with a 'Copy' button). At the bottom, there is a 'Back to access tokens' button.

Personal access tokens / New access token

Copy access token

Use this token as a password when you sign in from the Docker CLI client. [Learn more](#)
Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.

Access token description
jenkins-pipeline

Expires on
Never

Access permissions
Read & Write

To use the access token from your Docker CLI client:

1. Run
`$ docker login -u rajkumari2010` [Copy](#)

2. At the password prompt, enter the personal access token.
`dckr_pat_USXeer3J91FY1HGF3HoFuaLnkZI` [Copy](#)

[Back to access tokens](#)

. Add Docker Hub Credentials to Jenkins

Open Jenkins:

Jenkins



Manage Jenkins



Credentials



System



Global credentials



Add Credentials

Choose:

Kind:

Username with password

Enter:

Username:

your Docker Hub username

For example:

rajumari

For the password field, enter the **Docker Hub access token**, not your normal password.

Then:

ID:

dockerhub-credentials

Description:

Docker Hub Credentials

Click:

Create

Your Jenkins credential ID is now:

dockerhub-credentials

We will use this ID in the Jenkinsfile.

The screenshot shows the Docker Hub interface for a repository named 'rajkumari2010/jenkins-docker-demo'. The 'Tags' tab is selected, displaying a table with one tag, '1'. The table columns are 'Digest', 'OS/ARCH', 'Last pull', and 'Compressed size'. The tag '1' has a digest of '282bb3fbdf34', is for 'linux/amd64', was pulled 'less than 1 day' ago, and has a compressed size of '60.39 MB'. A 'Docker commands' section on the right shows the command 'docker pull rajkumari2010/jenkins-docker-demo:1' with a play button. The left sidebar shows the user's account 'rajkumari2010' and various repository management options.

Digest	OS/ARCH	Last pull	Compressed size
282bb3fbdf34	linux/amd64	less than 1 day	60.39 MB



Not secure 34.230.92.39:8080



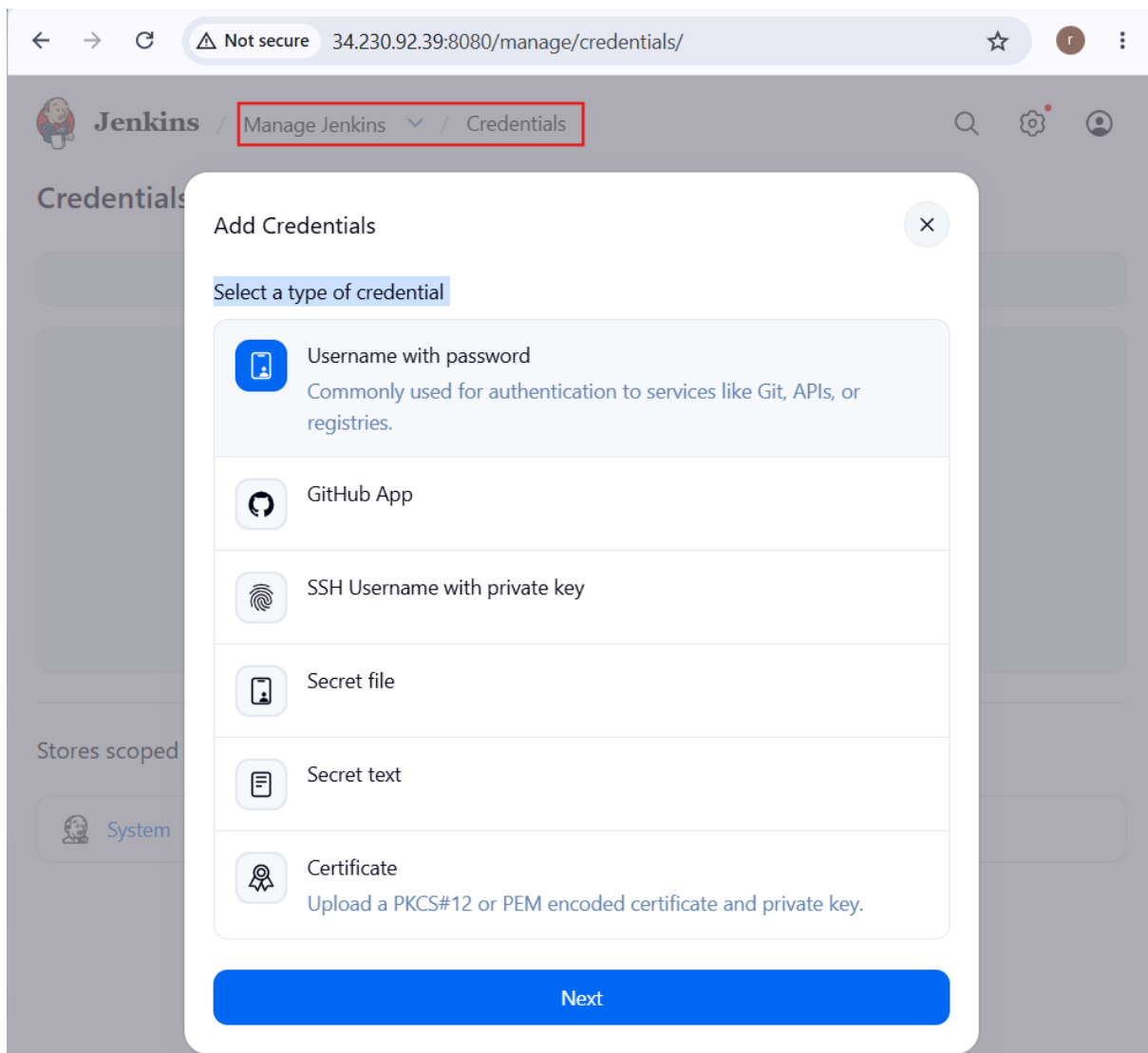
Getting Started

Jenkins is ready!

Your Jenkins setup is complete.

[Start using Jenkins](#)

Jenkins 2.568.2



← → ↻ ⚠ Not secure 34.230.92.39:8080/manage/credentials/ 🔑 ☆ 🧑

 **Jenkins** / Manage Jenkins ▾ / Credentials 🔍 ⚙️ 🧑

Credentials

Stores scoped to Jenkins system

 System

⏪ Add Username with password ⏩

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▾

Username
rajkumari2010

☐ Treat username as secret ?

Password
.....

ID ?
dockerhub-credentials

Description ?
DockerHub Credentials

Create

Credentials

+ Add Credentials



dockerhub-credentials rajkumari2010/*****

 System - Global - Docker Hub Credentials



Stores scoped to Jenkins



System

Domains: **Global**

5. Create a Sample Docker Project

For example, create:

jenkins-docker-demo

Project structure:

jenkins-docker-demo/

|

├── Dockerfile

├── index.html

└── Jenkinsfile

6. Create Dockerfile

Create:

vi Dockerfile

```
root@ip-172-31-30-131:~/jenkin-docker
FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

7. Create index.html

```
root@ip-172-31-30-131:~/jenkin-docker
<!DOCTYPE html>
<html>
<head>
  <title>Jenkins Docker Demo</title>
</head>
<body>
  <h1>Hello Rajkumari.....from Jenkins!</h1>
  <p>Docker image created and pushed using Jenkins Pipeline.</p>
</body>
</html>
```

8. Test Docker Build Manually

Before involving Jenkins, test your Dockerfile:

```
docker build -t jenkins-docker-demo:v1 .
```

Check:

```
docker images
```



```
root@ip-172-31-30-131:~/jenkin-docker
[root@ip-172-31-30-131 jenkins-docker]# vi Dockerfile
[root@ip-172-31-30-131 jenkins-docker]# vi index.html
[root@ip-172-31-30-131 jenkins-docker]# docker build -t jenkins-docker-demo:v1 .
[+] Building 0.8s (8/8) FINISHED
=> [internal] load build definition from Dockerfile                                0.1s
=> => transferring dockerfile: 212B                                              0.0s
=> [internal] load metadata for docker.io/library/nginx:latest                  0.4s
=> [auth] library/nginx:pull token for registry-1.docker.io                    0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> [internal] load build context                                                0.0s
=> => transferring context: 313B                                               0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:latest@sha256:b34848eff6db7       0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html                     0.1s
=> exporting to image                                                            0.0s
=> => exporting layers                                                            0.0s
=> => writing image sha256:04f7a31a23ca206f5d8b49dd699c17ded53908d43b6fc      0.0s
=> => naming to docker.io/library/jenkins-docker-demo:v1                     0.0s
[root@ip-172-31-30-131 jenkins-docker]# docker images
docker: 'images' is not a docker command.
See 'docker --help'
[root@ip-172-31-30-131 jenkins-docker]# docker images
REPOSITORY          TAG                 IMAGE ID            SIZE
jenkins-docker-demo v1                 04f7a3             162MB
docker-scan-demo    v2                 071c99             162MB
3a79a7              About an hour ago
571833381790.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo v1                 34b3ff             162MB
b37895              3 hours ago
docker-scan-demo    v1                 34b3ff             162MB
b37895              3 hours ago
dockerhub-scan-demo v1                 34b3ff             162MB
b37895              3 hours ago
rajkumari2010/docker-image-scan-demo v1                 34b3ff             162MB
b37895              3 hours ago
compose-demo-web    latest             646553             162MB
7f4622              13 hours ago
multi-stage-multi    v1                 f1941c             282MB
3ca17c              14 hours ago
multi-stage-single    v1                 b106e3             469MB
f4c768              14 hours ago
alpine-tomcat        1.0                96e1b5             227MB
0cb876              15 hours ago
my-nginx-running     v2                 384c40             162MB
c04463              17 hours ago
my-nginx-image        v1                 39c541             162MB
b70324              20 hours ago
redis                alpine             00c30d             119MB
df0ef8              8 days ago
[root@ip-172-31-30-131 jenkins-docker]# |
```

Create a container and run on port no 8081

```
root@ip-172-31-30-131:~/jenkin-docker
[root@ip-172-31-30-131 jenkins-docker]# docker run -d --name jenkins-docker-cont
-p 8081:80 jenkins-docker-demo:v1
32f6c9c2c568abd62ac68b4370d08429724bb010988d000440d9f1622aabaccc
[root@ip-172-31-30-131 jenkins-docker]# docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED
STATUS            PORTS              NAMES
32f6c9c2c568       jenkins-docker-demo:v1  "/docker-entrypoint..." 7 seconds ago
Up 5 seconds      0.0.0.0:8081->80/tcp, :::8081->80/tcp  jenkins-docker-cont
[root@ip-172-31-30-131 jenkins-docker]# |
```



Hello Rajkumari.....from Jenkins!

Docker image created and pushed using Jenkins Pipeline.

Test:

```
curl http://localhost:8080
```

If everything works:

```
docker stop jenkins-docker-demo
```

```
docker rm jenkins-docker-demo
```

```
root@ip-172-31-30-131:~/jenkin-docker
[root@ip-172-31-30-131 jenkins-docker]# docker run -d --name jenkins-docker-cont
-p 8081:80 jenkins-docker-demo:v1
32f6c9c2c568abd62ac68b4370d08429724bb010988d000440d9f1622aabaccc
[root@ip-172-31-30-131 jenkins-docker]# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                               NAMES
32f6c9c2c568   jenkins-docker-demo:v1             "/docker-entrypoint..." 7 seconds ago
Up 5 seconds   0.0.0.0:8081->80/tcp, :::8081->80/tcp jenkins-docker-cont
[root@ip-172-31-30-131 jenkins-docker]# docker stop 32f6c9c2c568abd62
32f6c9c2c568abd62
[root@ip-172-31-30-131 jenkins-docker]# docker rm 32f6c9c2c568abd62
32f6c9c2c568abd62
[root@ip-172-31-30-131 jenkins-docker]# |
```

9. Push Project to GitHub

Create a GitHub repository, for example:

```
jenkins-docker-demo
```

Then:

```
git init
```

Add files:

```
git add .
```

Commit:

```
git commit -m "Initial Docker Jenkins pipeline"
```

Add remote:

```
git remote add origin https://github.com/<username>/jenkins-docker-demo.git
```

Push:

```
git branch -M main
```

```
git push -u origin main
```

Your GitHub repository should contain:

Dockerfile

index.html

Jenkinsfile

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk ().*

1

General

Owner *

 **rajkumari-hub** ▾

Repository name *

jenkins-docker-demo

✔ jenkins-docker-demo is available.

Great repository names are short and memorable. How about [friendly-enigma](#)?

Description

Jenkins Docker CI/CD Demo

25 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

 **Public** ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

9. Push Project to GitHub

Create a GitHub repository, for example:

jenkins-docker-demo

Then:

git init

Add files:

git add .

Commit:

git commit -m "Initial Docker Jenkins pipeline"

Add remote:

git remote add origin https://github.com/<username>/jenkins-docker-demo.git

Push:

git branch -M main

git push -u origin main

Your GitHub repository should contain:

Dockerfile

index.html

Jenkinsfile

10. Create Jenkinsfile

This is the most important part.

Create:

vi Jenkinsfile

```

root@ip-172-31-30-131:~/jenkin-docker
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = 'rajkumari2010/jenkins-docker-demo'
        DOCKER_TAG = "${BUILD_NUMBER}"
    }

    stages {
        stage('Git Clone') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/rajkumari-hub/jenkins-docker-demo.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh '''
                    docker build \
                    -t ${DOCKER_IMAGE}:${DOCKER_TAG} .
                '''
            }
        }

        stage('Docker Login') {
            steps {
                withCredentials([
                    usernamePassword(
                        credentialsId: 'dockerhub-credentials',
                        usernameVariable: 'DOCKER_USERNAME',
                        passwordVariable: 'DOCKER_PASSWORD'
                    )
                ]) {
                    sh '''
                        echo "${DOCKER_PASSWORD}" | \
                        docker login \
                        -u "${DOCKER_USERNAME}" \
                        --password-stdin
                    '''
                }
            }
        }

        stage('Push Docker Image') {
            steps {
                sh '''
                    docker push ${DOCKER_IMAGE}:${DOCKER_TAG}
                '''
            }
        }
    }

    post {
        success {
            -- INSERT --
        }
    }
}

```

11. Understand the Jenkinsfile

Pipeline

```
pipeline {
```

This indicates that we are using a **Declarative Jenkins Pipeline**.

Agent

```
agent any
```

Jenkins can execute the pipeline on any available agent.

If you specifically want to use your Docker agent, you could use a label:

```
agent {
    label 'devops-node'
```

```
}
```

For your setup, if the Docker installation is on your devops-node, this is often better.

12. Environment Variables

```
environment {  
    DOCKER_IMAGE = 'rajkumari/jenkins-docker-demo'  
    DOCKER_TAG   = "${BUILD_NUMBER}"  
}
```

This defines:

DOCKER_IMAGE

DOCKER_TAG

BUILD_NUMBER is a Jenkins built-in variable.

Suppose Jenkins executes build:

Build #15

Then:

DOCKER_TAG = 15

The resulting image is:

rajkumari/jenkins-docker-demo:15

This is better than always using latest, because each Jenkins build gets its own image version.

13. Git Clone Stage

```
stage('Git Clone') {  
    steps {  
        git branch: 'main',  
        url: 'https://github.com/<github-username>/jenkins-docker-demo.git'
```

```
}  
}
```

Jenkins downloads the source code from GitHub.

The workspace will contain:

Dockerfile

index.html

Jenkinsfile

14. Build Docker Image Stage

```
stage('Build Docker Image') {  
    steps {  
        sh '''  
            docker build \  
            -t ${DOCKER_IMAGE}:${DOCKER_TAG} .  
            '''  
    }  
}
```

Jenkins executes:

```
docker build -t rajkumari/jenkins-docker-demo:15 .
```

The image is created locally on the Jenkins agent.

15. Docker Login Stage

```
withCredentials([  
    usernamePassword(  
        credentialsId: 'dockerhub-credentials',  
        usernameVariable: 'DOCKER_USERNAME',
```

```
        passwordVariable: 'DOCKER_PASSWORD'
    )
})
```

Jenkins retrieves the Docker Hub credentials securely.

Then:

```
echo "$DOCKER_PASSWORD" | docker login \
-u "$DOCKER_USERNAME" \
--password-stdin
```

This is preferable to putting the username/password directly into the Jenkinsfile.

16. Push Docker Image Stage

```
stage('Push Docker Image') {
    steps {
        sh '''
            docker push ${DOCKER_IMAGE}:${DOCKER_TAG}
        '''
    }
}
```

For build number 15:

```
docker push rajkumari/jenkins-docker-demo:15
```

The image goes to:

Docker Hub

↓

rajkumari/jenkins-docker-demo

↓

15


```

root@ip-172-31-30-131:~/jenkin-docker
[root@ip-172-31-30-131 jenkins-docker]# vi Jenkinsfile
[root@ip-172-31-30-131 jenkins-docker]# git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Jenkinsfile

nothing added to commit but untracked files present (use "git add" to track)
[root@ip-172-31-30-131 jenkins-docker]# git add .
[root@ip-172-31-30-131 jenkins-docker]# git commit -m "added jenkins"
[main 5996acf] added jenkins
Committer: root <root@ip-172-31-30-131.ec2.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 65 insertions(+)
create mode 100644 Jenkinsfile
[root@ip-172-31-30-131 jenkins-docker]# git push origin main
Username for 'https://github.com': rajkumari-hub
Password for 'https://rajkumari-hub@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 807 bytes | 807.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/rajkumari-hub/jenkins-docker-demo.git
   3470ee3..5996acf  main -> main
[root@ip-172-31-30-131 jenkins-docker]#

```

17. Create Jenkins Pipeline Job

Open Jenkins:

Jenkins Dashboard



New Item

Enter:

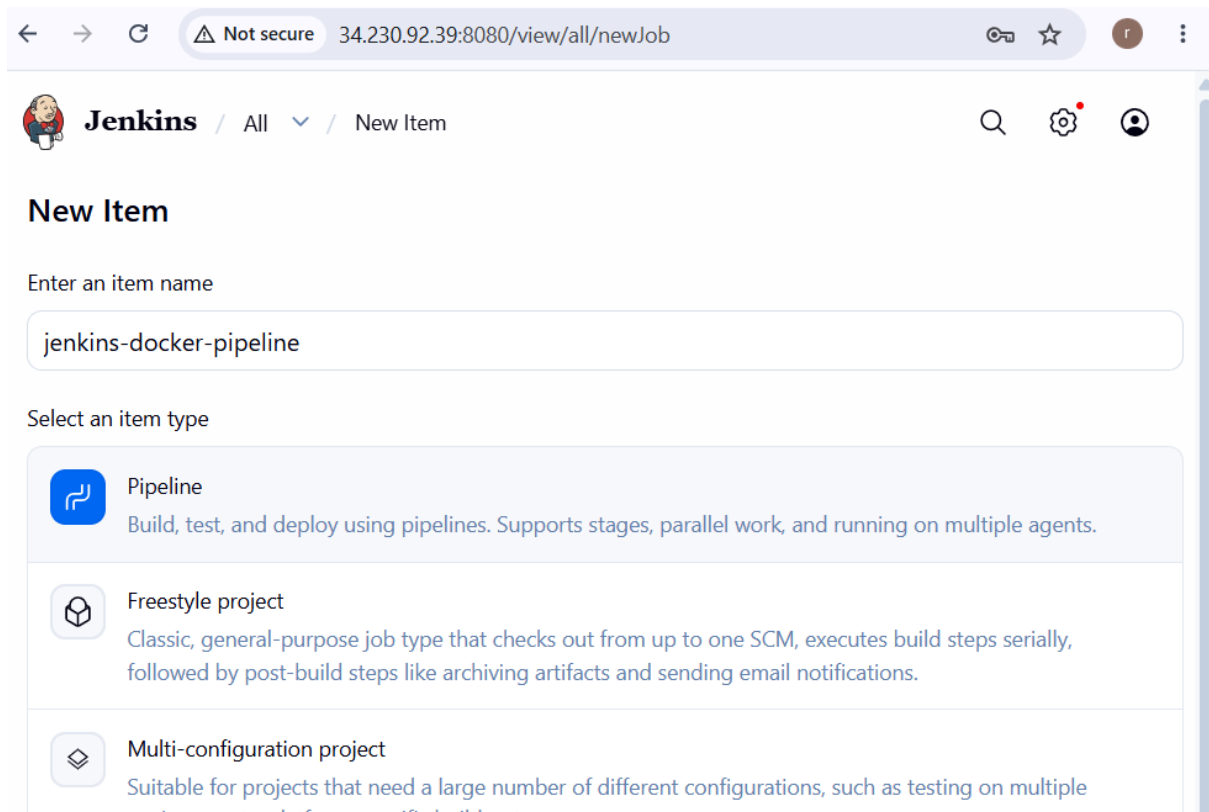
jenkins-docker-pipeline

Select:

Pipeline

Click:

OK



18. Configure Pipeline

Scroll to:

Pipeline

Select:

Definition:

Pipeline script from SCM

Select:

SCM:

Git

Repository URL:

`https://github.com/<github-username>/jenkins-docker-demo.git`

Branch:

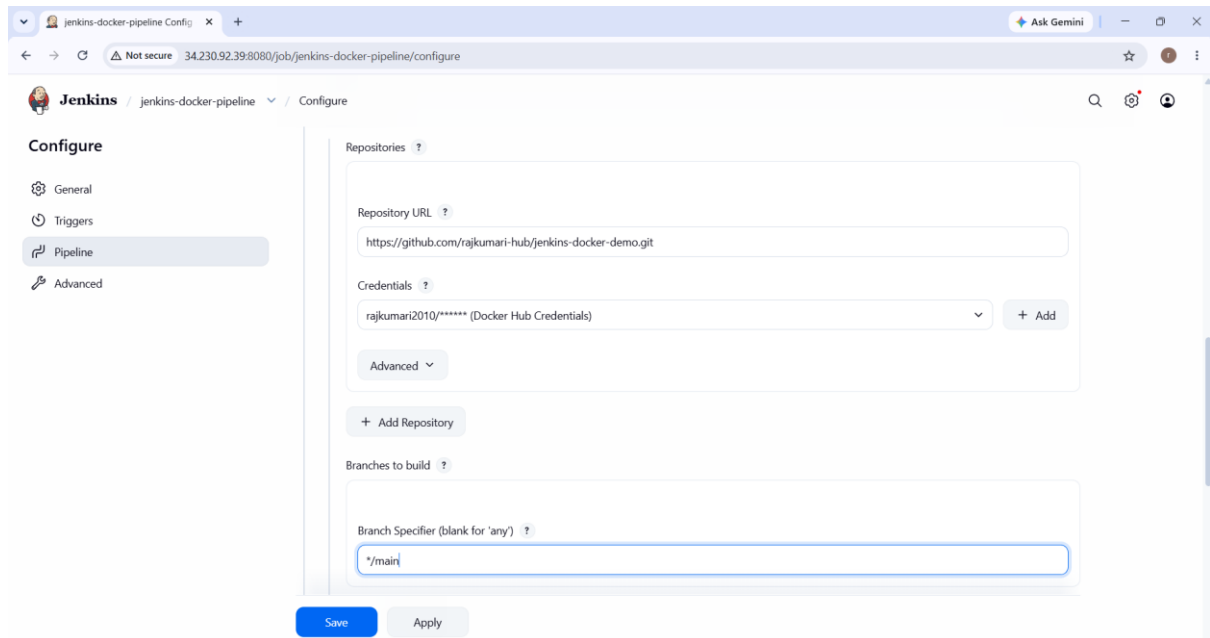
`*/main`

Script Path:

Jenkinsfile

Click:

Save



19. Run the Pipeline

Click:

Build Now

Jenkins will execute:

Git Clone



Build Docker Image



Docker Login



Push Docker Image

You should see:

[Pipeline] stage

[Pipeline] { (Git Clone)

[Pipeline] stage

[Pipeline] { (Build Docker Image)

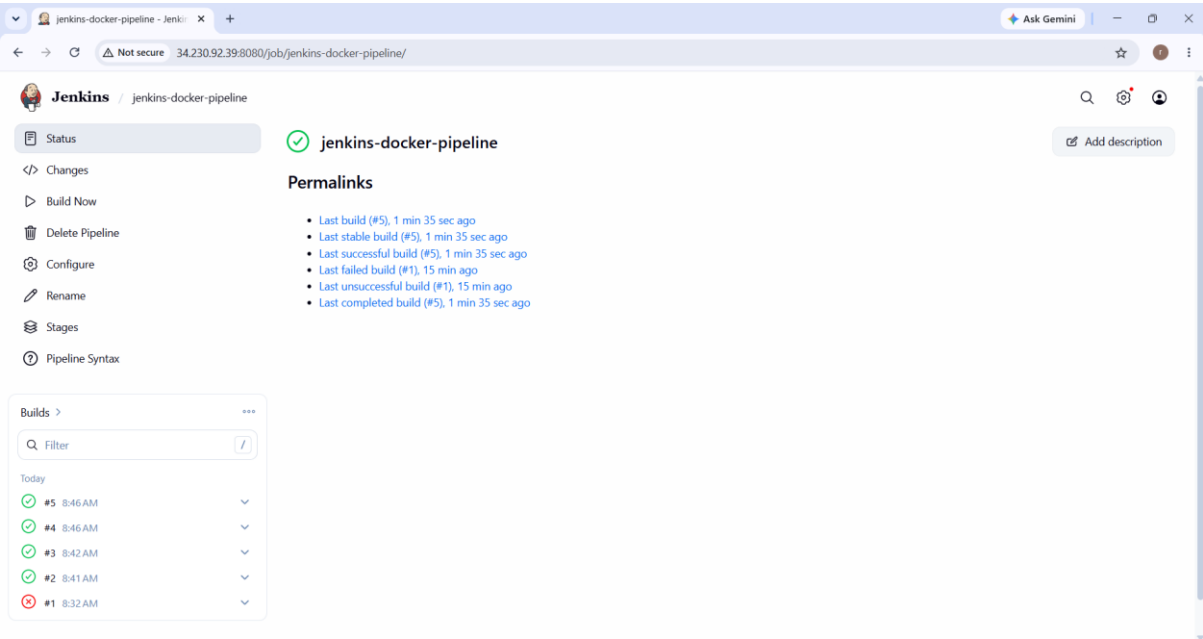
[Pipeline] stage

[Pipeline] { (Docker Login)

[Pipeline] stage

[Pipeline] { (Push Docker Image)

Finished: SUCCESS



jenkins-docker-pipeline #3 Con x + Ask Gemini

← → ↺ Not secure 34.230.92.39:8080/job/jenkins-docker-pipeline/3/console

Jenkins jenkins-docker-pipeline #3

Status
Changes
Console Output
Delete build #3
Get Build Data
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build

Console

Started by user rajkumar
Obtained Jenkinsfile from git https://github.com/rajkumar-hub/jenkins-docker-demo.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/jenkins-docker-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential dockerhub-credentials
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/jenkins-docker-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/rajkumar-hub/jenkins-docker-demo.git # timeout=10
Fetching upstream changes from https://github.com/rajkumar-hub/jenkins-docker-demo.git
> git --version # timeout=10
> git --version # 'git version 2.38.1'
using GIT_ASKPASS to set credentials Docker Hub Credentials
> git fetch --tags --force --progress -- https://github.com/rajkumar-hub/jenkins-docker-demo.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5996ac636b973b0b8db0ba0b7239f944ba58198 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 5996ac636b973b0b8db0ba0b7239f944ba58198 # timeout=10
Commit message: "added Jenkins"
> git rev-list --no-walk 5996ac636b973b0b8db0ba0b7239f944ba58198 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/jenkins-docker-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/rajkumar-hub/jenkins-docker-demo.git # timeout=10

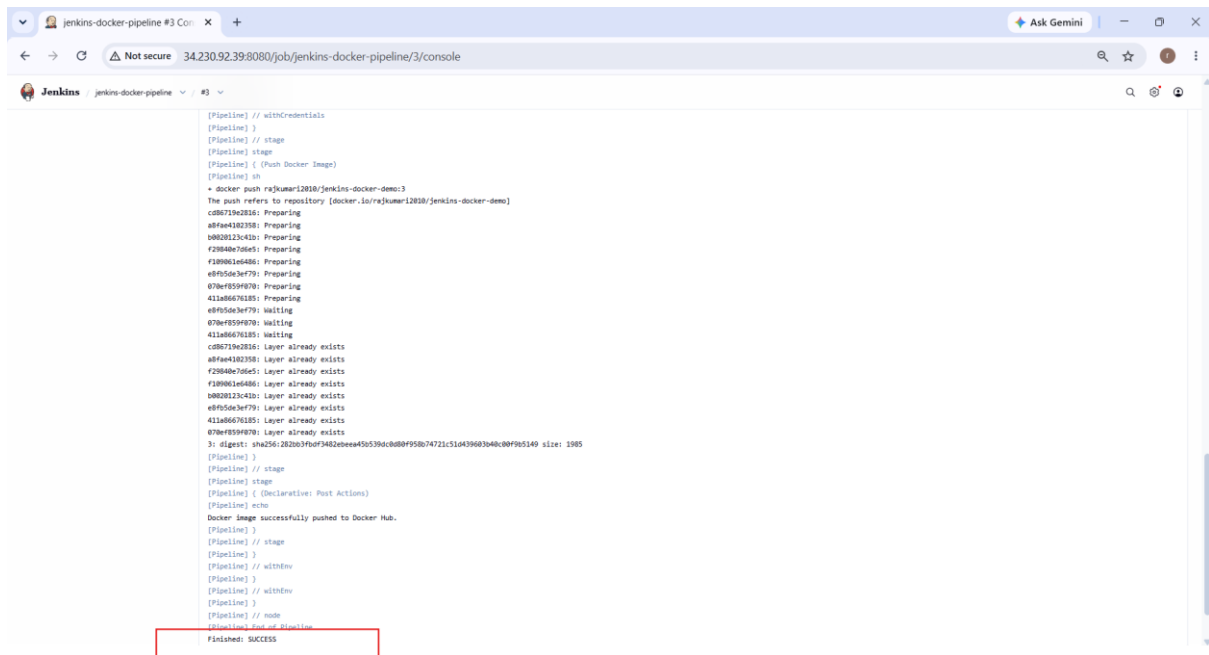
jenkins-docker-pipeline #3 Con x + Ask Gemini

← → ↺ Not secure 34.230.92.39:8080/job/jenkins-docker-pipeline/3/console

Jenkins jenkins-docker-pipeline #3

Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build

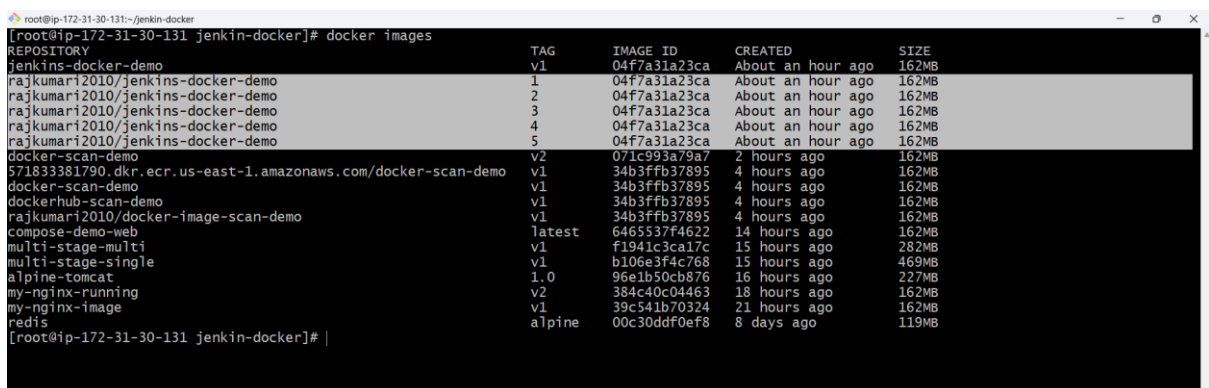
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential dockerhub-credentials
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/jenkins-docker-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/rajkumar-hub/jenkins-docker-demo.git # timeout=10
Fetching upstream changes from https://github.com/rajkumar-hub/jenkins-docker-demo.git
> git --version # timeout=10
> git --version # 'git version 2.38.1'
using GIT_ASKPASS to set credentials Docker Hub Credentials
> git fetch --tags --force --progress -- https://github.com/rajkumar-hub/jenkins-docker-demo.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5996ac636b973b0b8db0ba0b7239f944ba58198 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 5996ac636b973b0b8db0ba0b7239f944ba58198 # timeout=10
Commit message: "added Jenkins"
> git rev-list --no-walk 5996ac636b973b0b8db0ba0b7239f944ba58198 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/jenkins-docker-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/rajkumar-hub/jenkins-docker-demo.git # timeout=10
Fetching upstream changes from https://github.com/rajkumar-hub/jenkins-docker-demo.git
> git --version # timeout=10
> git --version # 'git version 2.38.1'
using GIT_ASKPASS to set credentials Docker Hub Credentials
> git fetch --tags --force --progress -- https://github.com/rajkumar-hub/jenkins-docker-demo.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5996ac636b973b0b8db0ba0b7239f944ba58198 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 5996ac636b973b0b8db0ba0b7239f944ba58198 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git checkout -D main # timeout=10
> git checkout -b main 5996ac636b973b0b8db0ba0b7239f944ba58198 # timeout=10



20. Verify the Image on Jenkins Server

SSH into the Docker/Jenkins agent and run:

docker images



21. Verify the Image in Docker Hub

Open:

[Docker Hub](https://hub.docker.com/u/rajakumar12030)

Go to:

Repositories



jenkins-docker-demo



Tags

You should see:

1

2

3

For example:

jenkins-docker-demo:15

The screenshot shows the Docker Hub interface for the repository 'rajkumari2010/jenkins-docker-demo'. The left sidebar contains navigation links: Repositories, Hardened Images, Collaborations, Settings (with sub-links for Default privacy, Notifications, Billing, Usage, Pulls, and Storage), and an 'Upgrade subscription' button. The main content area shows the repository details, including the name, last push time (4 minutes ago), size (60.4 MB), and download count (22). Below this is a 'Tags' section with a table listing the repository's tags. The table has columns for Tag, OS, Type, Pulled, and Pushed. It shows five tags (1-5) for the 'Image' type, all pulled 'less than 1 day' ago. The push times are 4, 4, 8, 8, and 10 minutes respectively. A 'See all' link is at the bottom of the table. To the right of the table is a 'Docker commands' section with a 'Public view' button and a code block containing the command 'docker push rajkumari2010/jenkins-docker-demo:tagname'. Further right is a 'buildcloud' advertisement for Docker Build Cloud.

Tag	OS	Type	Pulled	Pushed
5	linux	Image	less than 1 day	4 minutes
4	linux	Image	less than 1 day	4 minutes
3	linux	Image	less than 1 day	8 minutes
2	linux	Image	less than 1 day	8 minutes
1	linux	Image	less than 1 day	10 minutes